



Jonatha Rodrigues da Costa

Métodos Numéricos Aplicados à Engenharia: Abordagem com Python

Maracanaú
2026

Jonatha Rodrigues da Costa

Métodos Numéricos Aplicados à Engenharia: Abordagem com Python

Este livro trata sobre métodos numéricos aplicados à engenharia incluindo códigos em Python[®].

Instituto Federal de Educação, Ciência e Tecnologia do Ceará

Maracanaú
2026

Dedico este trabalho a Deus, a minha família, aos colegas acadêmicos e aos discentes, especialmente os discentes dos cursos de engenharia.

Agradecimentos

Meus agradecimentos mais profundos e expressivos são dirigidos àquele que me deu vida, que sustém-me em vida e que prometeu-me dar, pela eternidade, vida - Deus.

"Estudar é uma dádiva divina!"

Resumo

O livro “Métodos Numéricos Aplicados à Engenharia: Abordagem com Python” se destaca como uma valiosa contribuição para a comunidade acadêmica, abordando a aplicação prática de métodos numéricos no contexto da engenharia. Com um foco especial em Python®, o livro explora diversas bibliotecas populares, como NumPy, SciPy e Matplotlib, para facilitar a implementação e visualização de algoritmos numéricos.

Uma das principais inovações deste trabalho é a comparação sistemática dos resultados obtidos por diferentes métodos numéricos, proporcionando aos leitores uma compreensão crítica das técnicas disponíveis. Cada capítulo é estruturado para apresentar um problema específico de engenharia, seguido pela aplicação dos métodos numéricos, permitindo que os leitores vejam a transição da teoria à prática.

Além disso, todos os códigos utilizados nas demonstrações e exemplos são disponibilizados em um repositório no GitHub®, promovendo a transparência e a colaboração na comunidade acadêmica. Isso permite que estudantes e profissionais acessem, experimentem e aprimorem os algoritmos discutidos no livro, incentivando um aprendizado ativo e uma melhor assimilação dos conteúdos abordados.

Com uma abordagem didática e prática, “Métodos Numéricos Aplicados à Engenharia: Abordagem com Python” não apenas enriquece o conhecimento técnico dos leitores, mas também serve como um recurso valioso para a formação de futuros engenheiros e pesquisadores na utilização eficaz de ferramentas computacionais em suas atividades.

Sumário

Sumário	6
Lista de ilustrações	10
1 Introdução	11
1.1 Por que a engenharia não funciona sem métodos numéricos?	12
1.2 Abordagens de Métodos Numéricos	13
1.3 Computadores e Métodos Numéricos	14
1.4 Ambientes integrados a navegadores	15
1.4.1 Visual Studio Code	16
1.4.2 Spyder	17
1.5 Aprendizagem da Linguagem Python	18
2 Fundamentos Matemáticos: álgebra	19
2.1 Vetores	19
2.1.1 Definição de Vetor	19
2.1.2 Operações com Vetores	19
2.1.2.1 Adição e Subtração de Vetores	19
2.1.2.2 Multiplicação de Vetor por Escalar	19
2.1.2.3 Produto Escalar (ou Produto Interno)	20
2.1.2.4 Produto Vetorial (em 3 dimensões)	20
2.1.3 Norma de um Vetor	21
2.1.4 Propriedades dos Vetores	21
2.1.5 Script Python de operações básicas com vetores	21
2.2 Matrizes	22
2.2.1 Definição de Matriz	22
2.2.2 Operações com Matrizes	22
2.2.2.1 Adição e Subtração de Matrizes	22
2.2.2.2 Multiplicação de Matrizes	22
2.2.3 Script Python de operações com matrizes	23
2.3 Determinantes	23
2.3.1 Definição de Determinante	23
2.3.2 Propriedades dos Determinantes	24
2.3.3 Script Python para Cálculo de Determinantes e Inversas	24
2.4 Conclusão	24
3 Fundamentos Matemáticos: Funções	27
3.1 Classificação das Funções	27
3.1.1 Classificação quanto à expressão algébrica	27
3.1.2 Funções polinomiais	28
3.1.3 Resolução de equações cúbicas pelo método de Cardano	29
3.1.3.1 Transformação da equação cúbica	30
3.1.3.2 Interpretação do discriminante de Cardano	30

3.1.3.2.1	Caso 1: $\Delta > 0$	31
3.1.3.2.2	Caso 2: $\Delta = 0$	32
3.1.3.2.3	Caso 3: $\Delta < 0$	33
3.1.3.2.4	Resumo	34
3.1.3.3	Aplicação ao exemplo	34
3.1.4	Classificação quanto ao comportamento	41
3.1.5	Classificação quanto à simetria	41
3.1.6	Classificação quanto à correspondência	42
3.1.6.1	Função injetora	42
3.1.6.2	Função sobrejetora	43
3.1.6.3	Função bijetora	43
3.1.7	Classificação quanto à correspondência: síntese	44
3.1.8	Síntese das classificações	45
4	Fundamentos Matemáticos: cálculo	46
4.1	Conceitos de Cálculo: Limite, Derivada e Integral	46
4.2	Limites	46
4.2.1	Limites Laterais	46
4.2.2	Propriedades dos Limites	46
4.2.3	Script Python de operações básicas com limites	49
4.3	Derivadas	50
4.3.1	Propriedades das Derivadas	51
4.3.2	Script Python de operações básicas com derivadas	52
4.4	Integrais	53
4.4.1	Propriedades das Integrais	54
4.4.2	Script Python de operações básicas com integrais	55
4.5	Conclusão	57
5	Sistemas de Numeração	60
5.1	Conceituação de sistemas de numeração	60
5.1.1	Sistema Decimal (Base 10)	60
5.1.2	Sistema Binário (Base 2)	60
5.1.3	Sistema Octal (Base 8)	61
5.1.4	Sistema Hexadecimal (Base 16)	61
5.2	Conversão entre Sistemas de Numeração	61
5.2.1	Conversão de Decimal para Binário	61
5.2.2	Conversão de Binário para Decimal	61
5.2.3	Conversão de Decimal para Hexadecimal	62
5.2.4	Conversão de Hexadecimal para Decimal	62
5.2.5	Passos para a Conversão de um Número Fracionário de Base b para Decimal	62
5.3	Conversão de sistemas com Vírgula	63
5.3.1	Passos para a Conversão Fracionário para Decimal	63
5.3.2	Aplicação do procedimento de conversão	63
5.4	Script Python para Conversão entre sistemas	65
5.5	Conclusão	66

6	Aritmética de Ponto Flutuante e Erros Computacionais	68
6.1	Aritmética de Ponto Flutuante	68
6.2	Passo-a-passo generalizado para Conversão	68
6.3	Grafo da Conversão	70
6.4	Script em Python de conversão para Precisão Simples e Dupla	71
6.5	Armazenamento de Dados no Computador	72
6.5.1	Limitações na Representação	72
6.6	Erros de Arredondamento e de Truncamento	72
6.6.1	Erros de Arredondamento	72
6.6.2	Erros de Truncamento	72
6.7	Conclusão	73
7	Derivação Numérica	74
7.1	Conceituação de Derivação Numérica	74
7.2	Derivação progressiva	74
7.3	Derivação regressiva	75
7.4	Derivação Centrada	75
7.5	Erro na Aproximação da Derivação	76
7.5.1	Percepção numérica do erro de aproximação	76
7.6	Script básico Python para derivadas numéricas	77
7.7	Diferenças finitas por Série de Taylor	78
7.7.1	Aplicação Série de Taylor às Diferenças Finitas	78
7.7.2	Aproximação para a Derivada Primeira	78
7.7.3	Aproximação para a Derivada Segunda	79
7.7.4	Exemplos de Cálculo Numérico	79
7.8	Considerações Finais	80
8	Integração Numérica	81
8.1	Conceituação Integração Numérica	81
8.2	Método do Retângulo	82
8.2.1	Método do Retângulo Simples	82
8.2.2	Método do Retângulo Composto	82
8.3	Método do Ponto Central	83
8.3.1	Método do Ponto Central Simples	84
8.3.2	Método do Ponto Central Composto	84
8.4	Método do Trapézio	84
8.4.1	Método do Trapézio Simples	84
8.4.2	Método do Trapézio Composto	85
8.5	Regra de Simpson	86
8.5.1	Regra de Simpson 1/3	87
8.5.2	Regra de Simpson 3/8	87
8.6	Scripts Python para Métodos Integração	88
8.6.1	Integração Numérica Simples	88
8.6.2	Integração Numérica Composta	88
8.7	Considerações Finais	89

9	Métodos de Interpolação	90
9.1	Interpolação Polinomial de Lagrange	90
9.1.1	Script Python para interpolacao de Lagrange	91
9.2	Interpolação por Diferenças Divididas de Newton	92
9.2.1	Script Python para interpolacao de Newton	93
9.3	Comparativo entre os métodos de interpolação	94
10	Ajuste de curvas	95
10.1	Regressão Linear	95
10.2	Regressão Polinomial	95
10.3	Spline Linear	96
10.4	Spline Quadrático	96
10.5	Spline Cúbico	96

Lista de ilustrações

Figura 1 – Aplicação de Métodos Numéricos	13
Figura 2 – Ícones de ambientes computacionais matemáticos	15
Figura 3 – Logo da linguagem Python®	15
Figura 4 – IDEs via navegador de <i>web</i>	16
Figura 5 – Interface do Visual Studio Code®	17
Figura 6 – Interface do Spyder®	18
Figura 7 – Grafo da conversão pela IEEE 754/2008	70
Figura 8 – Equações de diferença para a derivada primeira	75
Figura 9 – Métodos de interação	82
Figura 10 – Comparação dos métodos de integração	82
Figura 11 – Método do Retângulo Simples	83
Figura 12 – Método do Retângulo Composto	83
Figura 13 – Método do Ponto Central Simples	84
Figura 14 – Método do Ponto Central Composto	85
Figura 15 – Método do Trapézio Simples	85
Figura 16 – Método do Trapézio Composto	86
Figura 17 – Interpolação de Lagrange	91
Figura 18 – Interpolação de Newton	93

1 Introdução

Métodos numéricos (MN) formam um conjunto poderoso de técnicas e algoritmos destinados à resolução de problemas complexos, com base em aproximações sucessivas. Essencialmente, esses métodos transformam equações matemáticas, muitas vezes difíceis de serem resolvidas analiticamente, em proposições mais simples, tratadas com operações aritméticas de baixa complexidade, mas alta precisão.

Esses métodos são amplamente aplicados em áreas onde soluções exatas não podem ser obtidas de maneira prática ou eficiente, como no caso de equações diferenciais, integrais e sistemas de equações lineares e não lineares. Sua relevância vem se consolidando cada vez mais em disciplinas como engenharia, física, ciência da computação, entre outras, sendo ferramentas fundamentais para a solução de problemas matemáticos. O rápido avanço das capacidades computacionais ao longo das últimas décadas tornou os métodos numéricos indispensáveis, permitindo a resolução de problemas que, há não muito tempo, seriam considerados intratáveis. Com o suporte de recursos computacionais acessíveis, a implementação de algoritmos de MN tornou-se mais eficiente e prática.

Conhecidos também como abordagens indiretas, os métodos numéricos baseiam-se na repetição de cálculos detalhados e progressivos, focados na obtenção de soluções aproximadas que convergem para resultados dentro de um intervalo de precisão aceitável. Embora as operações envolvidas sejam relativamente simples, a execução repetitiva pode gerar processos altamente intensivos do ponto de vista computacional. Nesse contexto, o conceito de **convergência** assume papel central: o objetivo dos MN é garantir que, com o aumento das iterações ou do refinamento do modelo, a solução se aproxime cada vez mais da resposta exata.

A análise numérica, ramo da matemática que estuda o comportamento desses métodos, é essencial para garantir a validade e a eficácia das soluções obtidas. Por meio dela, é possível avaliar a precisão, estabilidade e velocidade de convergência dos algoritmos aplicados. Além disso, a análise numérica oferece ferramentas para estudar e controlar os erros introduzidos pelas aproximações, como o **erro de truncamento** e o **erro de arredondamento**, que são inevitáveis em qualquer cálculo que envolva números com precisão finita. Assim, o desenvolvimento de métodos eficazes não se limita à busca de soluções; envolve também a compreensão profunda de como minimizar e gerenciar esses erros.

Os métodos numéricos, portanto, não fornecem soluções exatas no sentido clássico, mas sim **soluções aproximadas**, que são suficientemente precisas para a maioria das aplicações práticas. Esse caráter aproximativo é particularmente relevante no campo da engenharia, onde a modelagem de fenômenos físicos complexos resulta frequentemente em equações que não podem ser resolvidas de forma analítica. Como destacado por [Mathews et al. \(2000\)](#) e [Gilat e Subramaniam \(2008\)](#), a aplicação de métodos numéricos oferece uma solução computacionalmente viável para tais desafios, fornecendo respostas práticas e aplicáveis.

A importância dos métodos numéricos se amplia ainda mais quando consideramos as condições reais, onde frequentemente lidamos com dados incertos, medições com imprecisões e variabilidade nas condições experimentais. Ao empregar um método numérico, engenheiros e cientistas estão, na verdade, buscando soluções que, embora aproximadas, sejam suficientemente confiáveis para a tomada de decisões. Por isso, a **validação e verificação** das soluções numéricas, por meio de técnicas adequadas, são tão cruciais quanto o próprio processo de cálculo.

Além disso, o surgimento de ferramentas computacionais modernas, como MATLAB®, Python® e outras linguagens de programação, trouxe um nível de acessibilidade sem precedentes para a implementação dos métodos numéricos. Esses *softwares* facilitam a aplicação dos algoritmos, permitindo a resolução de problemas que seriam extremamente trabalhosos ou até inviáveis se feitos manualmente. Dessa forma, a combinação de uma base teórica sólida com o uso de ferramentas computacionais avançadas proporciona aos engenheiros e cientistas meios para abordar problemas de forma mais eficiente e segura.

Em síntese, os métodos numéricos são ferramentas indispensáveis para a resolução de problemas matemáticos em engenharia e em diversas outras disciplinas. Eles oferecem uma solução prática para questões complexas, transformando equações de difícil resolução em respostas viáveis por meio de aproximações precisas. O rigor na implementação e o profundo entendimento da análise de erros e da convergência são fundamentais para garantir que as soluções obtidas sejam confiáveis e aplicáveis na prática.

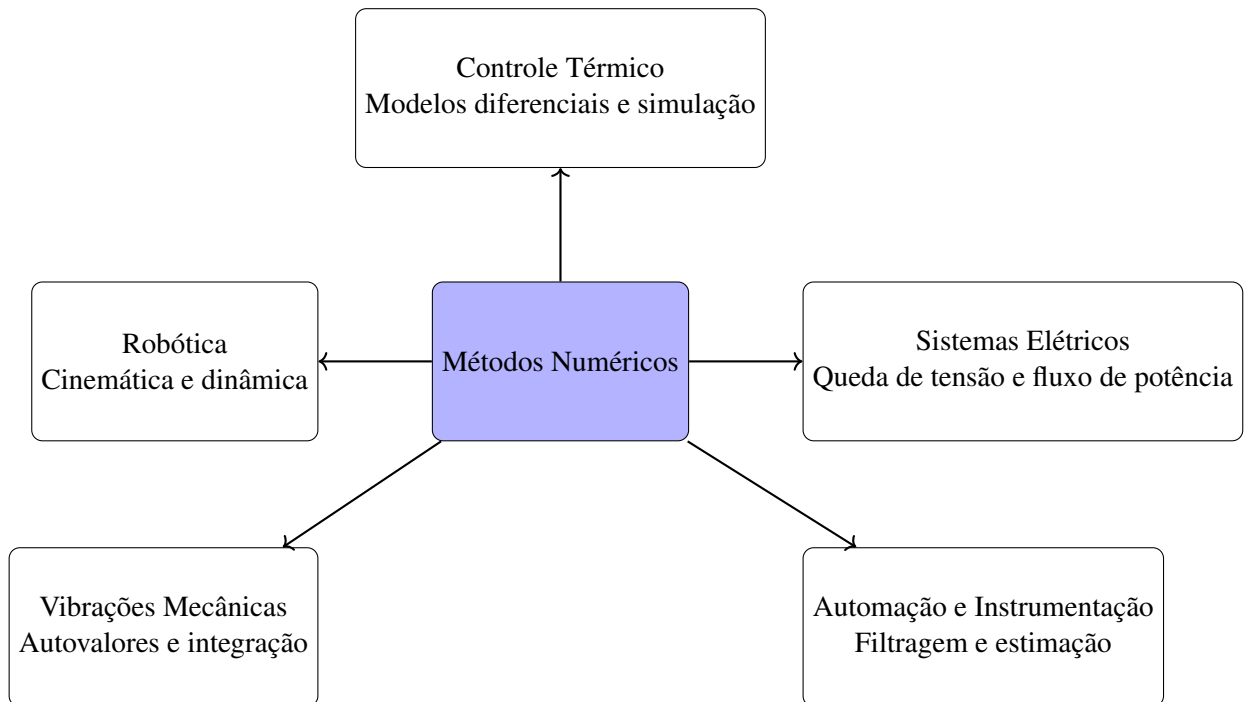
1.1 POR QUE A ENGENHARIA NÃO FUNCIONA SEM MÉTODOS NUMÉRICOS?

A engenharia lida com fenômenos que raramente admitem soluções analíticas. Sistemas reais apresentam não linearidades, parâmetros dependentes do tempo, grande número de variáveis, ruído, incerteza e restrições físicas. Os métodos numéricos tornam-se indispensáveis pelas seguintes razões:

1. **Modelos reais são não lineares.** Vibrações mecânicas não lineares, saturação de atuadores, histerese em materiais, turbulência e dinâmica térmica dependente do estado não possuem solução fechada em geral.
2. **Grande dimensão dos sistemas.** Estruturas com milhares de elementos, redes elétricas complexas, modelos térmicos tridimensionais e sistemas de controle multivariáveis dependem de álgebra matricial numérica.
3. **Dados experimentais substituem modelos ideais.** Sensores e atuadores fornecem sinais discretos e ruidosos; reconstruir derivadas, integrais e estados exige métodos numéricos estáveis.
4. **Simulação é economicamente superior ao protótipo físico.** Antes de construir um robô, uma ponte, uma planta térmica ou um conversor eletrônico, simular reduz custos e riscos.

Sem métodos numéricos, a engenharia seria restrita, incapaz de produzir previsões confiáveis e incapaz de lidar com problemas reais e multidimensionais. Na Figura 1 é apresentada uma visão basilar de aplicações de MN. Note que é apresentada a posição central dos métodos numéricos como ferramenta de cálculo aplicada em diferentes domínios da engenharia. Cada bloco evidencia um campo tradicional no qual tais métodos são essenciais: sistemas elétricos, por meio da análise de fluxo de potência; controle térmico, mediante modelos diferenciais; robótica, com resolução de problemas de cinemática e dinâmica; vibrações mecânicas, envolvendo autovalores e integração; e automação, por meio de filtragem e estimação de estados. O diagrama reforça a integração entre técnicas matemáticas e práticas consolidadas da engenharia.

Figura 1 – Aplicação de Métodos Numéricos



Fonte – Autor (2025)

1.2 ABORDAGENS DE MÉTODOS NUMÉRICOS

Como discutido em (CHAPRA; CANALE, 2016) e (PINTO, 2001), antes da utilização de MNs, a resolução de problemas e formulações matemáticas envolvia principalmente três abordagens, a saber:

- a) **Métodos puramente analíticos:** que, embora fossem frequentemente úteis, tinham limitações, oferecendo uma visão completa apenas para sistemas de geometria simples, baixa dimensão e baixa complexidade. Estes métodos eram práticos apenas para sistemas lineares, enquanto problemas do mundo real frequentemente envolvem sistemas não-lineares;
- b) **Métodos de soluções gráficas:** que caracterizavam o comportamento dos sistemas por meio de representações gráficas. Essas abordagens buscavam resolver problemas mais complexos, porém, muitas vezes, não produziam resultados precisos. Além disso, a implementação desses métodos podia ser desafiadora;
- c) **Métodos manuais com calculadoras:** utilizados para aplicar técnicas numéricas manualmente. No entanto, esses métodos frequentemente resultavam em inconsistências de cálculo devido a erros simples que ocorriam durante a execução de inúmeras tarefas manuais.

Observa-se, com facilidade, que esse processo tornava o trabalho de cálculo por MN demasiadamente exaustivo e passível de erros humanos. Contudo, ampliação do acesso aos computadores digitais mudou essa abordagem.

1.3 COMPUTADORES E MÉTODOS NUMÉRICOS

O avanço no desenvolvimento de computadores digitais rápidos e eficientes trouxe, por sua vez, um benefício já há muito desejado para a realização de cálculos repetitivos e reprocessamento de dados e sinais, conforme supracitado. Desse modo, além de fornecer uma capacidade de aumento de processamento e armazenamento de dados, a disponibilidade muito difundida dos computadores (especialmente dos computadores pessoais) representou uma influência significativa na resolução moderna de problemas de engenharia utilizando MN para tanto.

Convergente com isso, ambientes computacionais algébricos como MAPLE^{®1} e ambientes de desenvolvimento como o MATLAB[®] tem destaque desde o contexto acadêmico às aplicações industriais, sendo apontados dentre os preferidos por matemáticos e engenheiros quando necessitam resolver problemas que são viáveis apenas computacionalmente. Ambos os ambientes são fortemente utilizados na computação científica, como também na área de modelagem de sistemas e em atividades que necessitam de carga exaustiva de cálculo matemático, especialmente quando se utilizam aproximações por iterações.

Embora apresentem inúmeras vantagens em termos de facilidade e robustez, os referidos ambientes também apresentam características restritivas como: custo financeiro das licenças, código-fonte fechado² e alto custo computacional (consumo de tempo de processamento, memória e de energia na execução de um algoritmo ou *script*) na execução, conforme (SILVA et al., 2014). Essas características podem inviabilizar a utilização desses ambientes quando de aplicações com limitações de recursos.

Em face disto, *softwares* alternativos tem destacado-se com ampla utilização nas áreas supracitadas, conforme Costa (2017). Dentre esses:

- | | | |
|------------|------------|-----------|
| 1. MATLAB | 4. Sage | 7. Octave |
| 2. FreeMat | 5. Scilab | |
| 3. Maxima | 6. LabVIEW | |

O MATLAB[®] (ícone na Figura 2a) e o Octave[®] (ícone na Figura 2b) tem destaque especial por sua recursividade. No caso do primeiro, a quantidade de bibliotecas de estruturas de códigos já disponíveis em sua *toolbox* torna-o de elevada atratividade. O Octave^{®3}, por sua vez, sendo gratuito, multi plataforma e de interação próxima ao MATLAB[®], tem apresentado-se como uma alternativa à altura quando se trata de modelagem e simulação de sistemas no contexto de MN.

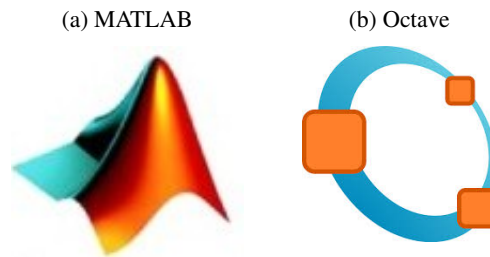
O MATLAB[®] & Octave[®] integram análise numérica, cálculo com matrizes, processamento de sinais e construção de gráficos em ambiente fácil de usar onde problemas e soluções são expressos somente como eles são escritos matematicamente, ao contrário da programação tradicional. Sendo que a ferramenta Octave, está disponível *on-line* e *off-line* sem custo, nos Sistemas Operacionais Microsoft Windows, GNU Linux, *Android* e afins.

¹ MAPLE: um sistema computacional comercial de uso genérico baseado em expressões algébricas, simbólicas, permitindo o desenho de gráficos em duas e três dimensões.

² CÓDIGO-FONTE FECHADO: *software* cujo o acesso, utilização, modificação ou redistribuição do código fonte, em quaisquer casos, são proibidos por quem tem os direitos sobre o código.

³ OCTAVE: Disponível também em simulação on-line <https://octave-online.net/>

Figura 2 – Ícones de ambientes computacionais matemáticos



Fonte: Autor, 2024

1.4 AMBIENTES INTEGRADOS A NAVEGADORES

Ambientes computacionais via navegadores de *internet* tem ganhado significativo espaço entre profissionais e acadêmicos. Nesse sentido, destaca-se a linguagem de programação Python® - ícone na Figura 3.

Figura 3 – Logo da linguagem Python®



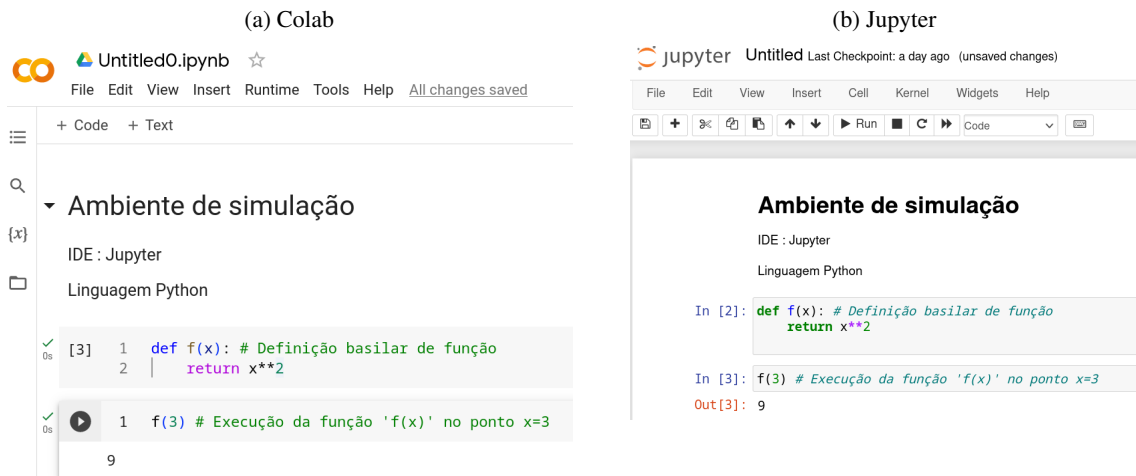
Fonte: Autor, 2024

Python® é uma linguagem de alto nível, interpretada e de utilização geral. Criada por Guido van Rossum, a linguagem tem sua primeira versão lançada em 1991. Desde então, a linguagem ganhou popularidade significativa devido à sua sintaxe simples e legibilidade, o que torna-a uma escolha popular tanto para iniciantes quanto para desenvolvedores experientes, conforme Hunt (2019).

Essa linguagem apresenta-se incorporada a um (*Integrated Development Environment* (ambiente de desenvolvimento integrado) (IDE) de empresas de serviços de *web* como a Google®, pelo recurso *google colab*, Figura 4a. Nesse mesmo contexto o Jupyter Notebook®, Figura 4b, apresenta-se também como uma aplicação *web* interativa que permite criar e compartilhar documentos que contêm código, equações, visualizações e texto narrativo.

Ambas são amplamente utilizados por cientistas de dados, pesquisadores acadêmicos, engenheiros e educadores para explorar dados, criar modelos, realizar análises estatísticas e compartilhar resultados de maneira interativa e fácil de entender. Além destas IDEs que integram esta linguagem, também são conhecidas outras símeis como: Spyder(<<https://www.spyder-ide.org>>), Visual Studio Code(<<https://code.visualstudio.com>>), GDBonline (<<https://www.onlinegdb.com>>), Replit (<<https://replit.com/>>), PyCharm (<<https://www.jetbrains.com/pycharm/>>) e Atom (<<https://atom.io/>>).

Em síntese, a utilização de IDEs *on-line* é útil para aprender Python®, por exemplo, porque oferece um ambiente de desenvolvimento prático, interativo e sem barreiras, permitindo que os acadêmicos

Figura 4 – IDEs via navegador de *web*

Fonte: Autor (2024)

se concentrem na lógica da programação e nas técnicas da linguagem, em vez de se preocuparem com a configuração de ambientes de desenvolvimento complicados. Além disso, também promovem a colaboração e a criação de conteúdo educacional interativo.

1.4.1 VISUAL STUDIO CODE

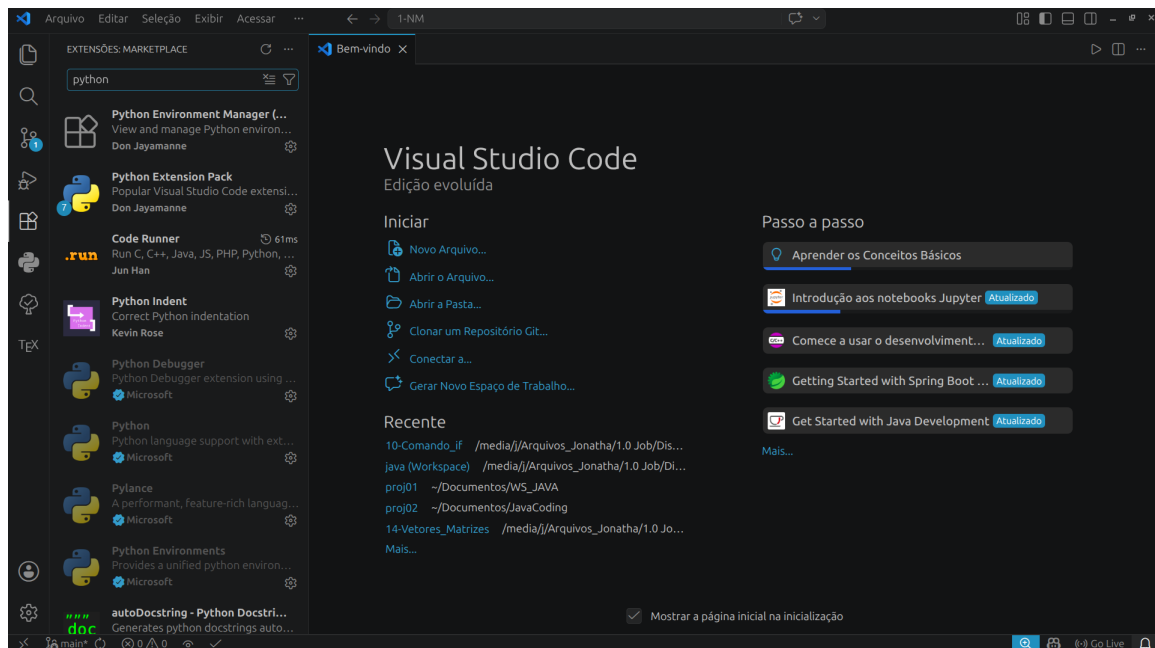
O Visual Studio Code (VS Code)[®], desenvolvido pela Microsoft[®], destaca-se atualmente como uma das principais IDEs utilizadas no desenvolvimento de aplicações computacionais. Trata-se de um editor de código-fonte leve, multiplataforma e altamente extensível, permitindo sua utilização em sistemas operacionais como Windows[®], Linux[®] e macOS[®].

A popularidade do VS Code decorre de sua interface intuitiva, elevada capacidade de personalização e integração com múltiplas linguagens de programação, dentre elas Python[®]. Conforme ilustrado na Figura 5, o ambiente oferece recursos avançados de edição de código, depuração, controle de versão e integração com terminais internos, tornando-o adequado tanto para aplicações acadêmicas quanto profissionais.

No contexto do desenvolvimento em Python[®], o VS Code apresenta vantagens relevantes, como a instalação de extensões específicas para análise de código, autocompletar inteligente (*IntelliSense*), execução de scripts, integração com ambientes virtuais e suporte nativo ao Jupyter Notebook[®]. Dessa forma, o ambiente proporciona maior produtividade no desenvolvimento de algoritmos, experimentos computacionais e aplicações científicas.

Outra característica importante refere-se à integração do VS Code com plataformas de computação em nuvem e sistemas de controle de versão, como o Git[®] e o GitHub[®]. Esses recursos favorecem práticas modernas de desenvolvimento colaborativo, permitindo o compartilhamento de códigos, rastreamento de alterações e gerenciamento eficiente de projetos computacionais. Além disso, o VS Code possibilita a utilização de extensões voltadas para ciência de dados, inteligência artificial e aprendizado de máquina, áreas nas quais Python[®] possui ampla aplicação. Nesse sentido, a ferramenta consolida-se como uma solução flexível e robusta para atividades de ensino, pesquisa e desenvolvimento tecnológico.

Figura 5 – Interface do Visual Studio Code®



Fonte: Autor (2026)

1.4.2 SPYDER

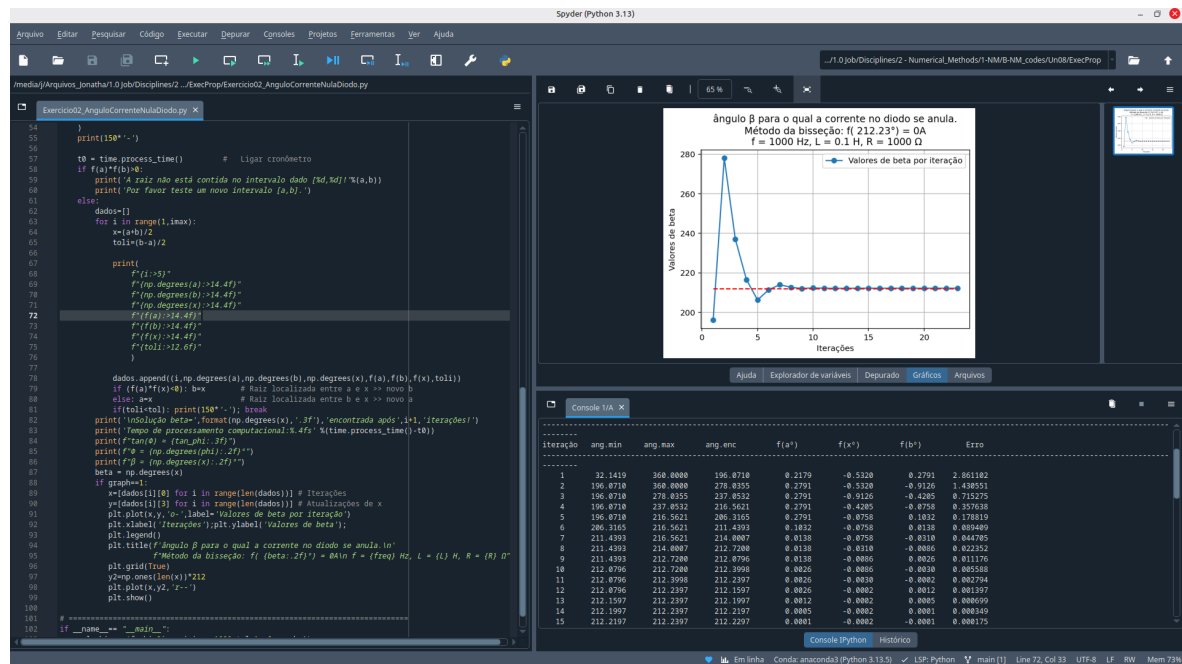
O Spyder® (*Scientific Python Development Environment*) consiste em uma IDE voltada especialmente para aplicações científicas e computacionais utilizando Python®. Desenvolvido com foco em pesquisa acadêmica, análise numérica e ciência de dados, o ambiente apresenta uma interface integrada semelhante a softwares matemáticos tradicionais, favorecendo a adaptação de estudantes, pesquisadores e engenheiros.

Conforme ilustrado na Figura 6, o Spyder integra em um único ambiente recursos como editor de código, console interativo, visualização de variáveis, depuração e execução de *scripts*, permitindo maior praticidade no desenvolvimento de aplicações científicas e educacionais.

Uma das principais vantagens do Spyder refere-se à sua integração com bibliotecas científicas amplamente utilizadas em Python®, como NumPy, SciPy, Matplotlib e Pandas. Tal característica possibilita a realização de cálculos numéricos, visualização gráfica de dados e processamento computacional de forma eficiente e intuitiva. Outro aspecto relevante consiste no explorador de variáveis (*Variable Explorer*), recurso que permite visualizar e manipular variáveis durante a execução do programa, facilitando atividades de depuração e análise de resultados. Esse mecanismo apresenta elevada importância em aplicações acadêmicas e laboratoriais, nas quais a interpretação de dados e a validação de modelos computacionais constituem etapas fundamentais.

O Spyder também oferece suporte à execução interativa de códigos em células, funcionalidade semelhante ao Jupyter Notebook®, permitindo que trechos específicos do programa sejam executados individualmente. Essa característica favorece experimentações computacionais, testes rápidos e análises iterativas em atividades de pesquisa científica. Além disso, a ferramenta apresenta integração com ambientes virtuais e gerenciadores de pacotes, contribuindo para a organização e reprodutibilidade de projetos computacionais. Sua distribuição conjunta ao pacote Anaconda® amplia ainda mais sua utilização

Figura 6 – Interface do Spyder®



Fonte: Autor (2026)

em cursos, laboratórios e aplicações voltadas à ciência de dados e engenharia.

Em síntese, o Spyder® consolida-se como uma IDE robusta e especializada para aplicações científicas em Python®, oferecendo recursos que favorecem o ensino, a pesquisa acadêmica e o desenvolvimento de soluções computacionais em diferentes áreas do conhecimento.

1.5 APRENDIZAGEM DA LINGUAGEM PYTHON

Um curso prático de Python® aplicado aos MNs está disponibilizado pelo autor deste trabalho através do Github® em <<https://github.com/jonathacosta/py2eng/tree/main/PyBasicCodes>>.

2 Fundamentos Matemáticos: álgebra

Neste capítulo, revisaremos os principais conceitos matemáticos que servem como base para os estudos de [MN](#). Estes incluem álgebra linear, além de algumas de suas propriedades essenciais. Estes tópicos são cruciais para a aplicação eficaz de métodos numéricos em problemas práticos de engenharia.

2.1 VETORES

Os vetores são elementos fundamentais na álgebra e na matemática aplicada, especialmente no estudo de métodos numéricos. Em geral, um vetor pode ser entendido como uma sequência de valores (ou elementos) dispostos em uma única linha (vetor linha) ou em uma única coluna (vetor coluna). Eles são amplamente utilizados em operações algébricas, análise numérica e computação científica.

2.1.1 DEFINIÇÃO DE VETOR

Um vetor é definido como uma lista ordenada de números, que podem representar quantidades físicas ou matemáticas. O vetor é representado por uma letra minúscula com uma seta em cima, como $\vec{v}$, ou, em notação matricial, por v .

Exemplo de vetor coluna:

$$\vec{v} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Exemplo de vetor linha:

$$\vec{w} = \begin{bmatrix} 1 & 3 & 5 \end{bmatrix}$$

2.1.2 OPERAÇÕES COM VETORES

2.1.2.1 Adição e Subtração de Vetores

A adição e subtração de vetores é realizada de forma componente a componente. Para que essas operações sejam possíveis, os vetores envolvidos devem ter a mesma dimensão.

Exemplo Numérico:

Sejam os vetores $\vec{a} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}$ e $\vec{b} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}$. Então:

$$\vec{a} + \vec{b} = \begin{bmatrix} 3+1 \\ 5+2 \\ 7+4 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 11 \end{bmatrix}$$

2.1.2.2 Multiplicação de Vetor por Escalar

A multiplicação de um vetor por um escalar α é realizada multiplicando cada componente do vetor pelo valor de α .

Exemplo Numérico:

Para o vetor $\vec{c} = \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix}$ e o escalar $\alpha = 3$, temos:

$$\alpha \cdot \vec{c} = 3 \cdot \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ -9 \\ 12 \end{bmatrix}$$

2.1.2.3 Produto Escalar (ou Produto Interno)

O produto escalar entre dois vetores $\vec{u} = \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix}$ e $\vec{v} = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}$ é dado pela soma do produto de seus elementos correspondentes. O produto escalar é representado por $\vec{u} \cdot \vec{v}$.

$$\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

Exemplo Numérico:

Se $\vec{u} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$ e $\vec{v} = \begin{bmatrix} 4 & 5 & 6 \end{bmatrix}$, então:

$$\vec{u} \cdot \vec{v} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32$$

2.1.2.4 Produto Vetorial (em 3 dimensões)

O produto vetorial é uma operação definida apenas em três dimensões, que resulta em um vetor perpendicular a ambos os vetores iniciais. Para dois vetores $\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$ e $\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$, o produto vetorial $\vec{u} \times \vec{v}$ é calculado como:

$$\vec{u} \times \vec{v} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} = \hat{i}(u_2 v_3 - u_3 v_2) - \hat{j}(u_1 v_3 - u_3 v_1) + \hat{k}(u_1 v_2 - u_2 v_1)$$

Exemplo Numérico:

Para $\vec{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ e $\vec{v} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$:

$$\begin{aligned} \vec{u} \times \vec{v} &= \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ 1 & 2 & 3 \\ 4 & 5 & 6 \end{vmatrix} = \hat{i}(2 \cdot 6 - 3 \cdot 5) - \hat{j}(1 \cdot 6 - 3 \cdot 4) + \hat{k}(1 \cdot 5 - 2 \cdot 4) \\ &= \hat{i}(12 - 15) - \hat{j}(6 - 12) + \hat{k}(5 - 8) \\ &= -3\hat{i} + 6\hat{j} - 3\hat{k} = \begin{bmatrix} -3 \\ 6 \\ -3 \end{bmatrix} \end{aligned}$$

2.1.3 NORMA DE UM VETOR

A norma (ou módulo) de um vetor $\vec{v} = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}$ é uma medida de seu comprimento, dada por:

$$\|\vec{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

Exemplo Numérico:

Para o vetor $\vec{v} = \begin{bmatrix} 3 & 4 \end{bmatrix}$, temos:

$$\|\vec{v}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

2.1.4 PROPRIEDADES DOS VETORES

Algumas propriedades úteis de vetores incluem:

- **Comutatividade da Adição:** $\vec{u} + \vec{v} = \vec{v} + \vec{u}$.
- **Associatividade da Adição:** $(\vec{u} + \vec{v}) + \vec{w} = \vec{u} + (\vec{v} + \vec{w})$.
- **Distributividade:** $\alpha(\vec{u} + \vec{v}) = \alpha\vec{u} + \alpha\vec{v}$.

2.1.5 SCRIPT PYTHON DE OPERAÇÕES BÁSICAS COM VETORES

O código a seguir ilustra como resolver os exemplos dos vetores utilizando Python® e a biblioteca *NumPy*:

```
import numpy as np

# Exemplo de Adicao de Vetores
a = np.array([3, 5, 7])
b = np.array([1, 2, 4])
soma = a + b
print("Soma de vetores a + b:", soma)

# Exemplo de Multiplicacao de Vetor por Escalar
c = np.array([2, -3, 4])
alpha = 3
produto_escalar = alpha * c
print("Multiplicacao do vetor c por escalar alpha:", produto_escalar)

# Exemplo de Produto Escalar (ou Produto Interno)
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
produto_interno = np.dot(u, v)
print("Produto escalar de u e v:", produto_interno)

# Exemplo de Produto Vetorial
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
produto_vetorial = np.cross(u, v)
print("Produto vetorial de u e v:", produto_vetorial)
```

```
# Exemplo de Norma de um Vetor
vetor = np.array([3, 4])
norma = np.linalg.norm(vetor)
print("Norma do vetor [3, 4]:", norma)
```

2.2 MATRIZES

As matrizes são um conceito fundamental na álgebra linear e são amplamente utilizadas em métodos numéricos para resolver sistemas de equações, representar transformações lineares e em diversas aplicações científicas e engenharias. Uma matriz é uma tabela retangular de números, organizada em linhas e colunas.

2.2.1 DEFINIÇÃO DE MATRIZ

Uma matriz A é representada como $A = [a_{ij}]$, em que a_{ij} é o elemento da matriz que está na i -ésima linha e j -ésima coluna. As dimensões de uma matriz são dadas pelo número de linhas e colunas, expressas como $m \times n$, em que m é o número de linhas e n é o número de colunas.

Exemplo de Matriz:

Uma matriz 2×3 :

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

2.2.2 OPERAÇÕES COM MATRIZES

2.2.2.1 Adição e Subtração de Matrizes

A adição e subtração de matrizes é realizada da mesma forma que a de vetores, ou seja, somando ou subtraindo os elementos correspondentes. Para que essas operações sejam possíveis, as matrizes devem ter as mesmas dimensões.

Exemplo Numérico:

Sejam as matrizes $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ e $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Então:

$$C = A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

2.2.2.2 Multiplicação de Matrizes

A multiplicação de matrizes é um pouco mais complexa. Para multiplicar duas matrizes A e B , o número de colunas de A deve ser igual ao número de linhas de B . O elemento c_{ij} da matriz resultante C é obtido pela soma dos produtos dos elementos da i -ésima linha de A pelos elementos da j -ésima coluna de B .

Exemplo Numérico:

$$\text{Seja } A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ e } B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}:$$

$$C = A \cdot B = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

2.2.3 SCRIPT PYTHON DE OPERAÇÕES COM MATRIZES

```
# Definicao de matrizes
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Exemplo de Adicao de Matrizes
C = A + B
print("Soma de matrizes A + B:\n", C)

# Exemplo de Subtracao de Matrizes
D = A - B
print("Subtracao de matrizes A - B:\n", D)

# Exemplo de Multiplicacao de Matrizes
E = np.dot(A, B)
print("Multiplicacao de matrizes A * B:\n", E)
```

2.3 DETERMINANTES

Os determinantes são uma ferramenta fundamental na álgebra linear, proporcionando informações importantes sobre as matrizes, como a invertibilidade, o volume e a solução de sistemas de equações lineares. Um determinante é um número associado a uma matriz quadrada, que pode ser calculado por diferentes métodos, dependendo da dimensão da matriz.

2.3.1 DEFINIÇÃO DE DETERMINANTE

Para uma matriz quadrada A de ordem n , o determinante é denotado como $\det(A)$ ou $|A|$. Para matrizes de 2×2 e 3×3 , as definições são as seguintes:

- Para uma matriz 2×2 :

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \det(A) = ad - bc$$

- Para uma matriz 3×3 :

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \Rightarrow \det(A) = a(ei - fh) - b(di - fg) + c(dh - eg)$$

Exemplo Numérico:

Para a matriz $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$:

$$\det(A) = (1)(4) - (2)(3) = 4 - 6 = -2$$

Para a matriz $B = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$:

$$\begin{aligned} \det(B) &= 1(1 \cdot 0 - 4 \cdot 6) - 2(0 \cdot 0 - 4 \cdot 5) + 3(0 \cdot 6 - 1 \cdot 5) = 1(0 - 24) - 2(0 - 20) + 3(0 - 5) \\ &= -24 + 40 - 15 = 1 \end{aligned}$$

2.3.2 PROPRIEDADES DOS DETERMINANTES

Os determinantes possuem várias propriedades importantes:

- **Determinante da Matriz Identidade:** $\det(I) = 1$, em que I é a matriz identidade.
- **Matriz Escalar:** Se A é uma matriz $n \times n$ e k é um escalar, então $\det(kA) = k^n \cdot \det(A)$.
- **Matriz Transposta:** $\det(A^T) = \det(A)$.
- **Multiplicação de Matrizes:** $\det(AB) = \det(A) \cdot \det(B)$.
- **Troca de Linhas:** Trocar duas linhas de uma matriz muda o sinal do determinante.
- **Linha Nula:** Se uma linha (ou coluna) de uma matriz é composta apenas por zeros, então $\det(A) = 0$.
- **Dependência Linear:** Se duas linhas (ou colunas) de uma matriz são iguais, então $\det(A) = 0$.

2.3.3 SCRIPT PYTHON PARA CÁLCULO DE DETERMINANTES E INVERSAS

Abaixo é apresentado um *script* em Python[®] utilizando a biblioteca NumPy para calcular os determinantes das matrizes mencionadas:

```
import numpy as np

# Definicao das matrizes
A = np.array([[1, 2], [3, 4]])
B = np.array([[1, 2, 3], [0, 1, 4], [5, 6, 0]])

# Exemplo de Determinante para matriz 2x2
det_A = np.linalg.det(A)
print("Determinante da matriz A:\n", det_A)

# Exemplo de Determinante para matriz 3x3
det_B = np.linalg.det(B)
print("Determinante da matriz B:\n", det_B)
```

2.4 CONCLUSÃO

A Álgebra fornece a base para diversos métodos numéricos, possibilitando manipular e resolver sistemas lineares, além de fornecer propriedades essenciais para simplificar cálculos. O entendimento das operações com matrizes, determinantes e propriedades algébricas é fundamental para o desenvolvimento de algoritmos numéricos eficientes. Códigos adicionais sobre cálculo explorando outras bibliotecas estão disponíveis no endereço <https://github.com/jonathacosta/NM/tree/main/NM_codes/Un01>

EXERCÍCIOS DE ÁLGEBRA LINEAR: VETORES, MATRIZES E DETERMINANTES

PARTE 1: VETORES

1. **Operações com Vetores** Considere os vetores $\vec{a} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$ e $\vec{b} = \begin{bmatrix} -1 \\ 4 \\ 2 \end{bmatrix}$. Calcule:

- a) $2\vec{a} + 3\vec{b}$
- b) O produto escalar $\vec{a} \cdot \vec{b}$
- c) O módulo de $\vec{a}$ e $\vec{b}$

2. **Vetores Ortogonais** Verifique se os vetores $\vec{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ e $\vec{v} = \begin{bmatrix} 4 \\ -8 \\ 0 \end{bmatrix}$ são ortogonais.

3. **Produto Vetorial** Encontre o produto vetorial entre os vetores $\vec{c} = \begin{bmatrix} 3 \\ -1 \\ 2 \end{bmatrix}$ e $\vec{d} = \begin{bmatrix} 1 \\ 4 \\ -1 \end{bmatrix}$.

PARTE 2: MATRIZES

4. **Operações com Matrizes** Considere as matrizes $A = \begin{bmatrix} 2 & 0 \\ -1 & 3 \end{bmatrix}$ e $B = \begin{bmatrix} 1 & 4 \\ 2 & -1 \end{bmatrix}$. Calcule:

- a) $A + B$
- b) $3A - 2B$
- c) O produto AB

5. **Matriz Transposta e Simetria** Dada a matriz $C = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{bmatrix}$:

- a) Encontre a matriz transposta C^T
- b) Verifique se C é uma matriz simétrica.

6. **Inversa de uma Matriz** Determine se a matriz $D = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$ é invertível. Caso seja, calcule D^{-1} .

PARTE 3: DETERMINANTES

7. Cálculo de Determinantes

Calcule o determinante das seguintes matrizes:

- a) $\begin{bmatrix} 3 & 2 \\ -1 & 4 \end{bmatrix}$
- b) $\begin{bmatrix} 1 & 0 & 2 \\ -3 & 5 & 1 \\ 4 & -2 & 3 \end{bmatrix}$

8. **Determinantes e Dependência Linear** Verifique se as colunas da matriz $E = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ são linearmente dependentes usando o determinante.

9. **Teorema de Laplace** Calcule o determinante da matriz $F = \begin{bmatrix} 2 & 0 & 1 \\ -1 & 3 & 4 \\ 0 & -2 & 5 \end{bmatrix}$, expandindo em relação à primeira linha.

3 Fundamentos Matemáticos: Funções

As funções constituem uma das principais ferramentas matemáticas para descrever relações entre grandezas. Em diferentes áreas da ciência e da Engenharia, fenômenos e sistemas são frequentemente representados por meio de funções, permitindo estabelecer relações entre variáveis, analisar comportamentos e construir modelos matemáticos.

Contudo, o conceito de função não se restringe à obtenção de valores a partir de uma expressão algébrica. Uma função estabelece uma relação entre conjuntos e possibilita investigar propriedades como **domínio, imagem, crescimento, decrescimento, simetria e comportamento assintótico**. Essas propriedades fornecem informações essenciais para a interpretação de modelos matemáticos e para a análise de fenômenos reais.

Neste capítulo, serão apresentados os fundamentos necessários para o estudo das funções. Inicialmente, serão discutidas suas principais classificações e as relações entre domínio, contradomínio e imagem. Em seguida, serão introduzidas diferentes classes de funções, suas propriedades e representações gráficas. Ao longo do capítulo, será dada atenção especial à interpretação da função como uma relação entre variáveis, e não apenas como uma expressão algébrica. Essa perspectiva será fundamental para os estudos posteriores de limites, derivadas e integrais.

3.1 CLASSIFICAÇÃO DAS FUNÇÕES

Antes de estudar propriedades específicas e métodos de análise de funções, é necessário compreender que elas podem ser classificadas segundo diferentes critérios. Uma mesma função pode pertencer simultaneamente a diferentes classes, pois cada classificação evidencia uma característica particular de sua estrutura ou de seu comportamento.

Neste sentido, as funções podem ser classificadas quanto à sua expressão algébrica, ao grau, ao comportamento, à simetria e à relação estabelecida entre os elementos do domínio, do contradomínio e da imagem.

3.1.1 CLASSIFICAÇÃO QUANTO À EXPRESSÃO ALGÉBRICA

De acordo com sua expressão algébrica, as funções podem ser classificadas em diferentes tipos. Entre os principais, destacam-se:

- **Função constante:** é expressa na forma

$$f(x) = c,$$

em que c é uma constante.

- **Função linear:** é expressa na forma

$$f(x) = ax,$$

em que a é uma constante.

- **Função afim:** é expressa na forma

$$f(x) = ax + b,$$

em que a e b são constantes.

- **Função quadrática:** é expressa na forma

$$f(x) = ax^2 + bx + c, \quad a \neq 0.$$

- **Função polinomial:** é expressa na forma

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

em que $a_n \neq 0$ e os expoentes de x são **inteiros não negativos**.

- **Função racional:** pode ser expressa como o quociente de dois polinômios:

$$f(x) = \frac{P(x)}{Q(x)}, \quad Q(x) \neq 0.$$

- **Função irracional:** apresenta a variável sob um radical ou com expoente racional, como

$$f(x) = \sqrt{x+1}.$$

- **Função potência:** é expressa na forma

$$f(x) = x^\alpha,$$

em que α é uma constante.

- **Função exponencial:** é expressa na forma

$$f(x) = a^x, \quad a > 0, \quad a \neq 1.$$

- **Função logarítmica:** é expressa na forma

$$f(x) = \log_a x, \quad a > 0, \quad a \neq 1.$$

- **Função trigonométrica:** envolve funções como

$$\sin(x), \quad \cos(x), \quad \tan(x).$$

3.1.2 FUNÇÕES POLINOMIAIS

As funções polinomiais possuem particular importância no estudo do Cálculo. Um polinômio de grau n pode ser expresso na forma

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0, \quad (1)$$

em que $a_n \neq 0$ e os expoentes da variável são inteiros não negativos.

O maior expoente de x cujo coeficiente é diferente de zero determina o **grau do polinômio**. Assim, temos:

- grau 0: função constante;
- grau 1: função afim;
- grau 2: função quadrática;
- grau 3: função cúbica;
- grau n : função polinomial de grau n .

Por exemplo, considere a função

$$f(x) = x^3 - 6x^2 + 9x + 1. \quad (2)$$

O maior expoente da variável é 3. Portanto, $f(x)$ é uma função polinomial de grau 3, também denominada **função cúbica**.

É importante observar que os expoentes da variável em um polinômio devem ser inteiros não negativos. Por exemplo,

$$f(x) = x^3 - 6x^2 + 9x + 1$$

é uma função polinomial, enquanto

$$g(x) = x^3 + \frac{2}{x}$$

não é polinomial, pois

$$\frac{2}{x} = 2x^{-1}.$$

Entretanto, a função $g(x)$ pode ser escrita como

$$g(x) = \frac{x^4 + 2}{x},$$

sendo, portanto, uma **função racional**.

3.1.3 RESOLUÇÃO DE EQUAÇÕES CÚBICAS PELO MÉTODO DE CARDANO

As funções polinomiais de terceiro grau, também denominadas funções cúbicas, possuem a forma

$$f(x) = ax^3 + bx^2 + cx + d, \quad a \neq 0.$$

A determinação de suas raízes consiste em resolver a equação

$$ax^3 + bx^2 + cx + d = 0. \quad (3)$$

Uma das técnicas clássicas para a resolução de equações cúbicas é o **método de Cardano**, desenvolvido no século XVI. O método permite transformar uma equação cúbica geral em uma equação denominada **cúbica reduzida**, da forma

$$y^3 + py + q = 0. \quad (4)$$

A partir dessa forma, as raízes podem ser determinadas por meio da fórmula de Cardano.

3.1.3.1 Transformação da equação cúbica

Considere a equação cúbica

$$ax^3 + bx^2 + cx + d = 0.$$

Inicialmente, dividimos toda a equação por a , obtendo

$$x^3 + \frac{b}{a}x^2 + \frac{c}{a}x + \frac{d}{a} = 0.$$

Para eliminar o termo quadrático, fazemos a substituição

$$x = y - \frac{b}{3a}. \quad (5)$$

Após a substituição e a simplificação, obtém-se uma equação da forma

$$y^3 + py + q = 0,$$

em que

$$p = \frac{3ac - b^2}{3a^2} \quad (6)$$

e

$$q = \frac{27a^2d - 9abc + 2b^3}{27a^3}. \quad (7)$$

A equação reduzida

$$y^3 + py + q = 0$$

é a forma fundamental utilizada no método de Cardano.

3.1.3.2 Interpretação do discriminante de Cardano

Para a equação cúbica reduzida

$$y^3 + py + q = 0,$$

o método de Cardano utiliza o discriminante

$$\Delta = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3. \quad (8)$$

O sinal de Δ fornece informações importantes sobre a quantidade e a natureza das raízes reais da equação. Existem três situações fundamentais:

Valor de Δ	Número de raízes reais distintas
$\Delta > 0$	1
$\Delta = 0$	2 ou 1
$\Delta < 0$	3

(9)

Cada caso possui uma interpretação específica.

3.1.3.2.1 Caso 1: $\Delta > 0$

Quando

$$\Delta > 0,$$

a equação cúbica possui **uma única raiz real** e duas raízes complexas conjugadas.

Nesse caso,

$$-\frac{q}{2} + \sqrt{\Delta}$$

e

$$-\frac{q}{2} - \sqrt{\Delta}$$

são números reais distintos. Assim, a fórmula de Cardano fornece diretamente a raiz real:

$$y = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}. \quad (10)$$

As outras duas raízes são complexas conjugadas.

Esse é exatamente o caso da função estudada neste capítulo,

$$f(x) = x^3 - 6x^2 + 9x + 1.$$

Após a substituição

$$x = y + 2,$$

obtivemos

$$y^3 - 3y + 3 = 0,$$

para a qual

$$p = -3, \quad q = 3.$$

Consequentemente,

$$\Delta = \left(\frac{3}{2}\right)^2 + \left(\frac{-3}{3}\right)^3 = \frac{9}{4} - 1 = \frac{5}{4} > 0.$$

Portanto, a função possui uma única raiz real.

3.1.3.2.2 Caso 2: $\Delta = 0$

Quando

$$\Delta = 0,$$

a equação possui raízes reais, mas ocorre uma **multiplicidade**. Em outras palavras, pelo menos duas das três raízes coincidem.

Nesse caso, a equação pode apresentar:

- uma raiz real simples e uma raiz real dupla; ou
- uma única raiz real tripla.

Para compreender esse resultado, considere

$$\Delta = 0.$$

Então,

$$\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3 = 0.$$

A fórmula de Cardano assume uma forma simplificada e as raízes podem ser determinadas sem a necessidade de números complexos.

Por exemplo, considere

$$y^3 - 3y + 2 = 0.$$

Nesse caso,

$$p = -3, \quad q = 2.$$

Logo,

$$\Delta = \left(\frac{2}{2}\right)^2 + \left(\frac{-3}{3}\right)^3 = 1 - 1 = 0.$$

A equação pode ser fatorada como

$$y^3 - 3y + 2 = (y - 1)^2(y + 2).$$

Portanto, suas raízes são

$$y_1 = 1, \quad y_2 = 1, \quad y_3 = -2.$$

Assim, existem duas raízes reais distintas:

$y = 1, \text{ com multiplicidade } 2$
--

e

$$y = -2, \text{ com multiplicidade } 1.$$

Quando, além disso, $p = q = 0$, temos

$$y^3 = 0,$$

e a única raiz é

$$y = 0,$$

com multiplicidade 3.

Portanto, quando $\Delta = 0$, a equação possui **duas raízes reais distintas, sendo uma delas múltipla, ou uma única raiz real tripla.**

3.1.3.2.3 Caso 3: $\Delta < 0$

Quando

$$\Delta < 0,$$

a equação cúbica possui **três raízes reais distintas.**

Esse caso é particularmente interessante porque a fórmula de Cardano, quando escrita diretamente com radicais reais, conduz a expressões envolvendo raízes quadradas de números negativos.

De fato, como

$$\Delta < 0,$$

temos

$$\sqrt{\Delta} \in \mathbb{C}.$$

Consequentemente, a expressão

$$\sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}$$

envolve números complexos, embora o resultado final das três soluções seja completamente real.

Esse fenômeno é conhecido como **casus irreducibilis**.

Nesse caso, embora seja possível utilizar a fórmula de Cardano no conjunto dos números complexos, uma abordagem mais conveniente para determinar as três raízes reais consiste, frequentemente, em utilizar uma representação trigonométrica da solução.

3.1.3.2.4 Resumo

O discriminante de Cardano permite antecipar a natureza das raízes da equação cúbica reduzida

$$y^3 + py + q = 0.$$

Sua interpretação pode ser resumida por

Δ	Raízes reais distintas	Natureza das raízes
$\Delta > 0$	1	1 real e 2 complexas conjugadas
$\Delta = 0$	1 ou 2	raízes reais múltiplas
$\Delta < 0$	3	3 reais distintas

(11)

Portanto, antes de aplicar a fórmula de Cardano, é conveniente calcular Δ . O seu sinal fornece uma primeira informação sobre a estrutura das soluções e orienta a escolha do procedimento algébrico mais adequado.

3.1.3.3 Aplicação ao exemplo

Considere a função polinomial

$$f(x) = x^3 - 6x^2 + 9x + 1. \quad (12)$$

Para determinar suas raízes, devemos resolver

$$x^3 - 6x^2 + 9x + 1 = 0. \quad (13)$$

Comparando a Equação 13 com a forma geral

$$ax^3 + bx^2 + cx + d = 0,$$

temos

$$a = 1, \quad b = -6, \quad c = 9, \quad d = 1.$$

Utilizando a substituição dada pela Equação 5,

$$x = y - \frac{b}{3a},$$

obtemos

$$x = y - \frac{-6}{3} = y + 2.$$

Portanto,

$$x = y + 2. \quad (14)$$

Substituindo na equação original,

$$(y+2)^3 - 6(y+2)^2 + 9(y+2) + 1 = 0.$$

Expandindo os termos,

$$y^3 + 6y^2 + 12y + 8 - 6(y^2 + 4y + 4) + 9y + 18 + 1 = 0.$$

Assim,

$$y^3 + 6y^2 + 12y + 8 - 6y^2 - 24y - 24 + 9y + 18 + 1 = 0.$$

Agrupando os termos semelhantes,

$$y^3 + (6-6)y^2 + (12-24+9)y + (8-24+18+1) = 0.$$

Consequentemente,

$$y^3 - 3y + 3 = 0.$$

Obtivemos, portanto, a cúbica reduzida

$$y^3 - 3y + 3 = 0, \tag{15}$$

para a qual

$$p = -3 \quad \text{e} \quad q = 3.$$

Fórmula de Cardano

Para a equação reduzida

$$y^3 + py + q = 0,$$

define-se o discriminante de Cardano por

$$\Delta = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3. \tag{16}$$

No exemplo,

$$p = -3 \quad \text{e} \quad q = 3.$$

Logo,

$$\Delta = \left(\frac{3}{2}\right)^2 + \left(\frac{-3}{3}\right)^3.$$

Portanto,

$$\Delta = \frac{9}{4} - 1 = \frac{5}{4}.$$

Como

$$\Delta > 0,$$

a equação possui uma raiz real e duas raízes complexas conjugadas.

Pela fórmula de Cardano, a raiz real é

$$y = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}. \quad (17)$$

Substituindo $q = 3$ e $\Delta = 5/4$,

$$y = \sqrt[3]{-\frac{3}{2} + \frac{\sqrt{5}}{2}} + \sqrt[3]{-\frac{3}{2} - \frac{\sqrt{5}}{2}}.$$

Assim,

$$y = \frac{1}{2} \left[\sqrt[3]{-3 + \sqrt{5}} + \sqrt[3]{-3 - \sqrt{5}} \right]. \quad (18)$$

Como $x = y + 2$, obtemos a raiz real

$$x_1 = 2 + \frac{1}{2} \left[\sqrt[3]{-3 + \sqrt{5}} + \sqrt[3]{-3 - \sqrt{5}} \right]. \quad (19)$$

Numericamente,

$$x_1 \approx 1.468.$$

Determinação das demais raízes

Até este ponto, utilizamos a fórmula de Cardano para determinar uma raiz real da equação

$$y^3 - 3y + 3 = 0.$$

Obtivemos

$$y = \sqrt[3]{-\frac{3}{2} + \frac{\sqrt{5}}{2}} + \sqrt[3]{-\frac{3}{2} - \frac{\sqrt{5}}{2}}.$$

Como a equação é cúbica, ela possui três raízes no conjunto dos números complexos, contando suas multiplicidades. Como, neste caso,

$$\Delta = \frac{5}{4} > 0,$$

sabemos que uma dessas raízes é real e que as outras duas são complexas conjugadas.

A questão que surge é: **como obter essas duas raízes a partir da solução de Cardano?**

A ideia central.

Na fórmula de Cardano, aparecem duas raízes cúbicas. Para facilitar a notação, escrevemos

$$u = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} \quad (20)$$

e

$$v = \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}. \quad (21)$$

No exemplo,

$$u = \sqrt[3]{-\frac{3}{2} + \frac{\sqrt{5}}{2}}$$

e

$$v = \sqrt[3]{-\frac{3}{2} - \frac{\sqrt{5}}{2}}.$$

A fórmula de Cardano fornece inicialmente

$$y_1 = u + v.$$

Entretanto, uma raiz cúbica complexa não possui apenas um valor. Para um número complexo não nulo, existem três números que, elevados ao cubo, produzem esse mesmo número.

É justamente essa propriedade que permite obter as outras duas soluções.

As raízes cúbicas da unidade.

Considere a equação

$$z^3 = 1.$$

Uma solução evidente é

$$z = 1.$$

As outras duas soluções são números complexos. Elas são

$$\omega = -\frac{1}{2} + \frac{\sqrt{3}}{2}i \quad (22)$$

e

$$\omega^2 = -\frac{1}{2} - \frac{\sqrt{3}}{2}i.$$

Esses três números,

$$1, \quad \omega, \quad \omega^2,$$

são chamados de **raízes cúbicas da unidade**, pois

$$1^3 = \omega^3 = (\omega^2)^3 = 1.$$

Além disso,

$$1 + \omega + \omega^2 = 0.$$

Essa propriedade será utilizada para construir as três soluções da equação cúbica.

Construção das três soluções.

A primeira solução é obtida diretamente pela fórmula de Cardano:

$$y_1 = u + v.$$

Para obter a segunda solução, multiplicamos u por ω e v por ω^2 :

$$y_2 = \omega u + \omega^2 v.$$

Finalmente, para obter a terceira solução, fazemos a troca correspondente:

$$y_3 = \omega^2 u + \omega v.$$

Portanto, as três soluções da equação reduzida são

$$\boxed{\begin{aligned} y_1 &= u + v, \\ y_2 &= \omega u + \omega^2 v, \\ y_3 &= \omega^2 u + \omega v. \end{aligned}} \quad (23)$$

Essa é a ideia fundamental: as raízes cúbicas da unidade permitem gerar, a partir da expressão obtida por Cardano, as três raízes da equação cúbica.

Determinação da primeira raiz.

No exemplo,

$$u = \frac{1}{2} \sqrt[3]{-3 + \sqrt{5}}$$

e

$$v = \frac{1}{2} \sqrt[3]{-3 - \sqrt{5}}.$$

Assim,

$$y_1 = u + v$$

e, portanto,

$$y_1 = \frac{1}{2} \left[\sqrt[3]{-3 + \sqrt{5}} + \sqrt[3]{-3 - \sqrt{5}} \right].$$

Como a substituição utilizada anteriormente foi

$$x = y + 2,$$

temos

$$x_1 = 2 + \frac{1}{2} \left[\sqrt[3]{-3 + \sqrt{5}} + \sqrt[3]{-3 - \sqrt{5}} \right]. \quad (24)$$

Numericamente,

$$\boxed{x_1 \approx 1,468}.$$

Determinação da segunda raiz.

A segunda solução da equação reduzida é

$$y_2 = \omega u + \omega^2 v.$$

Substituindo

$$\omega = -\frac{1}{2} + \frac{\sqrt{3}}{2}i$$

e

$$\omega^2 = -\frac{1}{2} - \frac{\sqrt{3}}{2}i,$$

obtemos

$$y_2 = \left(-\frac{1}{2} + \frac{\sqrt{3}}{2}i \right) u + \left(-\frac{1}{2} - \frac{\sqrt{3}}{2}i \right) v.$$

Distribuindo os termos,

$$y_2 = -\frac{u}{2} + \frac{\sqrt{3}}{2}ui - \frac{v}{2} - \frac{\sqrt{3}}{2}vi.$$

Agrupando as partes real e imaginária,

$$y_2 = -\frac{u+v}{2} + \frac{\sqrt{3}}{2}(u-v)i.$$

Como $x = y + 2$,

$$x_2 = 2 - \frac{u+v}{2} + \frac{\sqrt{3}}{2}(u-v)i.$$

Substituindo os valores de u e v ,

$$\boxed{x_2 \approx 2,266 + 0,562i}.$$

Determinação da terceira raiz.

A terceira solução da equação reduzida é

$$y_3 = \omega^2 u + \omega v.$$

Substituindo as expressões de ω e ω^2 ,

$$y_3 = \left(-\frac{1}{2} - \frac{\sqrt{3}}{2}i\right)u + \left(-\frac{1}{2} + \frac{\sqrt{3}}{2}i\right)v.$$

Agrupando as partes real e imaginária,

$$y_3 = -\frac{u+v}{2} - \frac{\sqrt{3}}{2}(u-v)i.$$

Voltando para a variável x ,

$$x_3 = 2 - \frac{u+v}{2} - \frac{\sqrt{3}}{2}(u-v)i.$$

Numericamente,

$$x_3 \approx 2,266 - 0,562i.$$

Observe que

$$x_3 = \overline{x_2},$$

isto é, x_3 é o conjugado complexo de x_2 . Isso não é uma coincidência. Como a equação possui coeficientes reais, toda raiz complexa não real deve ocorrer acompanhada de sua conjugada.

Resultado final.

As três raízes da equação

$$x^3 - 6x^2 + 9x + 1 = 0$$

são, aproximadamente,

$$\begin{aligned} x_1 &\approx 1,468, \\ x_2 &\approx 2,266 + 0,562i, \\ x_3 &\approx 2,266 - 0,562i. \end{aligned}$$

(25)

Portanto, no conjunto dos números reais, a função

$$f(x) = x^3 - 6x^2 + 9x + 1$$

possui uma única raiz:

$$x \approx 1,468.$$

As outras duas raízes são complexas conjugadas e, portanto, não aparecem como pontos de interseção do gráfico da função com o eixo x no plano cartesiano real.

3.1.4 CLASSIFICAÇÃO QUANTO AO COMPORTAMENTO

Uma função também pode ser classificada de acordo com seu comportamento em determinado intervalo.

Uma função f é **crescente** em um intervalo I quando, para quaisquer $x_1, x_2 \in I$, temos

$$x_1 < x_2 \implies f(x_1) < f(x_2). \quad (26)$$

Analogamente, uma função f é **decrecente** em um intervalo I quando

$$x_1 < x_2 \implies f(x_1) > f(x_2). \quad (27)$$

No Cálculo Diferencial, o comportamento crescente ou decrescente de uma função pode ser analisado por meio de sua primeira derivada. Em condições adequadas,

$$f'(x) > 0$$

indica que a função é crescente, enquanto

$$f'(x) < 0$$

indica que a função é decrescente.

A segunda derivada permite analisar a concavidade da função. Quando

$$f''(x) > 0,$$

a função apresenta concavidade para cima. Quando

$$f''(x) < 0,$$

a função apresenta concavidade para baixo.

3.1.5 CLASSIFICAÇÃO QUANTO À SIMETRIA

Uma função é denominada **par** quando

$$f(-x) = f(x). \quad (28)$$

Nesse caso, o gráfico da função é simétrico em relação ao eixo y .

Por exemplo,

$$f(x) = x^2$$

é uma função par.

Uma função é denominada **ímpar** quando

$$f(-x) = -f(x). \quad (29)$$

Nesse caso, o gráfico apresenta simetria em relação à origem.

Por exemplo,

$$f(x) = x^3$$

é uma função ímpar.

Quando uma função não satisfaz nenhuma dessas duas condições, ela é classificada como **nem par nem ímpar**.

3.1.6 CLASSIFICAÇÃO QUANTO À CORRESPONDÊNCIA

Considere uma função

$$f : A \rightarrow B.$$

Nessa representação, o conjunto A é denominado **domínio** da função, enquanto B é denominado **contradomínio**. O conjunto dos valores efetivamente assumidos pela função é denominado **imagem** e é representado por

$$\text{Im}(f) = \{f(x); x \in A\}.$$

A relação entre domínio, contradomínio e imagem permite classificar uma função como injetora, sobrejetora ou bijetora.

3.1.6.1 Função injetora

Uma função $f : A \rightarrow B$ é denominada **injetora** quando elementos distintos do domínio possuem imagens distintas. Formalmente,

$$x_1 \neq x_2 \implies f(x_1) \neq f(x_2). \quad (30)$$

De forma equivalente, uma função é injetora quando

$$f(x_1) = f(x_2) \implies x_1 = x_2. \quad (31)$$

Em termos geométricos, uma função real de uma variável é injetora quando qualquer reta horizontal intercepta seu gráfico em, no máximo, **um ponto**.

Por exemplo, considere

$$f(x) = 2x + 1.$$

Se $f(x_1) = f(x_2)$, então

$$2x_1 + 1 = 2x_2 + 1,$$

o que implica

$$x_1 = x_2.$$

Portanto, $f(x) = 2x + 1$ é uma função injetora.

3.1.6.2 Função sobrejetora

Uma função $f : A \rightarrow B$ é denominada **sobrejetora** quando todo elemento do contradomínio é imagem de pelo menos um elemento do domínio. Formalmente,

$$\forall y \in B, \quad \exists x \in A \text{ tal que } f(x) = y. \quad (32)$$

Em outras palavras, uma função é sobrejetora quando sua imagem coincide com seu contradomínio:

$$\text{Im}(f) = B. \quad (33)$$

Por exemplo, considere

$$f : \mathbb{R} \rightarrow \mathbb{R}, \quad f(x) = x^3.$$

Para qualquer $y \in \mathbb{R}$, podemos escolher

$$x = \sqrt[3]{y}.$$

Assim,

$$f(x) = (\sqrt[3]{y})^3 = y.$$

Portanto, todo elemento do contradomínio é atingido pela função, e consequentemente

$f : \mathbb{R} \rightarrow \mathbb{R}, \quad f(x) = x^3 \text{ é sobrejetora.}$
--

3.1.6.3 Função bijetora

Uma função é denominada **bijetora** quando é simultaneamente injetora e sobrejetora.

$$f \text{ é bijetora} \iff f \text{ é injetora e sobrejetora.} \quad (34)$$

Uma função bijetora estabelece uma correspondência um a um entre os elementos do domínio e do contradomínio. Nesse caso, cada elemento do contradomínio é imagem de exatamente um elemento do domínio.

Por exemplo, considere

$$f : \mathbb{R} \rightarrow \mathbb{R}, \quad f(x) = 2x + 1.$$

A função é injetora, pois

$$f(x_1) = f(x_2) \implies x_1 = x_2.$$

Além disso, dado qualquer $y \in \mathbb{R}$, existe

$$x = \frac{y-1}{2}$$

tal que

$$f(x) = 2\left(\frac{y-1}{2}\right) + 1 = y.$$

Logo, a função é sobrejetora. Portanto,

$$\boxed{f(x) = 2x + 1 \text{ é uma função bijetora de } \mathbb{R} \text{ em } \mathbb{R}.}$$

3.1.7 CLASSIFICAÇÃO QUANTO À CORRESPONDÊNCIA: SÍNTESE

As três classificações podem ser resumidas da seguinte maneira:

- **Injetora:** elementos distintos do domínio possuem imagens distintas.
- **Sobrejetora:** todo elemento do contradomínio pertence à imagem da função.
- **Bijetora:** é simultaneamente injetora e sobrejetora.

A relação entre essas propriedades pode ser representada por

$$\boxed{\text{bijetora} \iff \text{injetora} \wedge \text{sobrejetora}}. \quad (35)$$

É importante destacar que a classificação como injetora, sobrejetora ou bijetora depende não apenas da expressão da função, mas também do domínio e do contradomínio especificados.

Por exemplo, a função

$$f(x) = x^2$$

não é injetora de $\mathbb{R}$ em $\mathbb{R}$, pois

$$f(-1) = f(1) = 1.$$

Entretanto, se restringirmos seu domínio para $[0, +\infty)$, a função passa a ser injetora. Além disso, sua sobrejetividade também depende do contradomínio escolhido.

Portanto, ao analisar uma função, deve-se sempre considerar a representação

$$f : A \rightarrow B,$$

e não apenas a expressão algébrica $f(x)$.

Tabela 1 – Principais classificações de funções.

Critério	Classificações
Expressão algébrica	constante, linear, afim, quadrática, polinomial, racional, irracional, potência, exponencial, logarítmica e trigonométrica
Grau	grau 0, grau 1, grau 2, grau 3, ..., grau n
Comportamento	crescente, decrescente ou constante
Concavidade	concavidade para cima ou para baixo
Simetria	par, ímpar ou nem par nem ímpar
Correspondência	injetora, sobrejetora ou bijetora

3.1.8 SÍNTESE DAS CLASSIFICAÇÕES

A classificação de uma função depende da propriedade que está sendo analisada. A Tabela 1 apresenta uma síntese das principais classificações estudadas neste capítulo.

Uma mesma função pode apresentar diferentes classificações simultaneamente. Por exemplo, uma função pode ser polinomial, de grau 3, crescente em determinado intervalo, ímpar e injetora. Cada classificação descreve um aspecto específico da função.

Assim, a análise completa de uma função exige a consideração de sua expressão algébrica, de seu domínio e contradomínio e de suas propriedades algébricas e geométricas.

4 Fundamentos Matemáticos: cálculo

Neste capítulo, revisaremos os principais conceitos matemáticos que servem como base para os estudos de [MN](#). Estes incluem cálculo diferencial e integral, além de algumas de suas propriedades essenciais. Estes tópicos são cruciais para a aplicação eficaz de métodos numéricos em problemas práticos de engenharia.

4.1 CONCEITOS DE CÁLCULO: LIMITE, DERIVADA E INTEGRAL

O Cálculo de Limite, Derivada e Integral é um ramo fundamental da matemática, amplamente utilizado para modelar e analisar fenômenos que envolvem variações contínuas. Em problemas de engenharia e outras ciências aplicadas, ele permite descrever tanto mudanças instantâneas quanto acumulações ao longo do tempo ou de espaço, sendo crucial para a resolução de problemas envolvendo movimento, fluxo, taxas de variação, e otimização.

4.2 LIMITES

Os **limites** desempenham um papel central no cálculo e na análise matemática, sendo a base para a definição de conceitos como derivadas e integrais. Um limite é o valor que uma função ou uma sequência "tende a" quando a variável independente se aproxima de um certo ponto.

Formalmente, o limite de uma função $f(x)$ conforme x se aproxima de um valor a é denotado na Equação 36:

$$\lim_{x \rightarrow a} f(x) = L \quad (36)$$

Isso significa que, à medida que x se aproxima de a , os valores de $f(x)$ se aproximam de L . Os limites são essenciais para entender o comportamento de funções em pontos específicos, especialmente em casos onde a função pode não estar definida ou pode ter um comportamento indefinido, como divisões por zero.

4.2.1 LIMITES LATERAIS

O conceito de **limite lateral** surge quando estamos interessados em entender o comportamento de uma função ao se aproximar de um ponto por um lado específico (esquerda ou direita). Denotamos o limite lateral à direita de a como $\lim_{x \rightarrow a^+} f(x)$ e à esquerda como $\lim_{x \rightarrow a^-} f(x)$.

Para que o limite $\lim_{x \rightarrow a} f(x)$ exista, é necessário que os limites laterais sejam iguais, conforme definido na Equação 37:

$$\lim_{x \rightarrow a^+} f(x) = \lim_{x \rightarrow a^-} f(x) \quad (37)$$

4.2.2 PROPRIEDADES DOS LIMITES

Os limites possuem diversas propriedades importantes que facilitam seu cálculo em diferentes contextos:

- **Propriedade de soma ou subtração**, conforme a Equação 38:

$$\lim_{x \rightarrow a} [f(x) \pm g(x)] = \lim_{x \rightarrow a} f(x) \pm \lim_{x \rightarrow a} g(x) \quad (38)$$

Calcule os seguintes limites, aplicando a propriedade de soma ou subtração:

- a) $\lim_{x \rightarrow 2} [x^2 + 3x]$
- b) $\lim_{x \rightarrow 4} [\sqrt{x} - x]$
- c) $\lim_{x \rightarrow 1} [\ln x + 1]$

Soluções:

a)

$$\begin{aligned} \lim_{x \rightarrow 2} [x^2 + 3x] &= \lim_{x \rightarrow 2} x^2 + \lim_{x \rightarrow 2} 3x \\ &= 4 + 6 \\ &= 10 \end{aligned}$$

b)

$$\begin{aligned} \lim_{x \rightarrow 4} [\sqrt{x} - x] &= \lim_{x \rightarrow 4} \sqrt{x} - \lim_{x \rightarrow 4} x \\ &= 2 - 4 \\ &= -2 \end{aligned}$$

c)

$$\begin{aligned} \lim_{x \rightarrow 1} [\ln x + 1] &= \lim_{x \rightarrow 1} \ln x + \lim_{x \rightarrow 1} 1 \\ &= 0 + 1 \\ &= 1 \end{aligned}$$

- **Propriedade do produto**, expressa pela Equação 39:

$$\lim_{x \rightarrow a} [f(x) \cdot g(x)] = \lim_{x \rightarrow a} f(x) \cdot \lim_{x \rightarrow a} g(x) \quad (39)$$

Calcule os seguintes limites, aplicando a propriedade do produto:

- a) $\lim_{x \rightarrow 3} [(x + 1) \cdot 2x]$
- b) $\lim_{x \rightarrow 2} [x \cdot x^2]$
- c) $\lim_{x \rightarrow 0} [\sin x \cdot x]$

Soluções:

a)

$$\begin{aligned}\lim_{x \rightarrow 3} [(x+1) \cdot 2x] &= \lim_{x \rightarrow 3} (x+1) \cdot \lim_{x \rightarrow 3} 2x \\ &= 4 \cdot 6 \\ &= 24\end{aligned}$$

b)

$$\begin{aligned}\lim_{x \rightarrow 2} [x \cdot x^2] &= \lim_{x \rightarrow 2} x \cdot \lim_{x \rightarrow 2} x^2 \\ &= 2 \cdot 4 \\ &= 8\end{aligned}$$

c)

$$\begin{aligned}\lim_{x \rightarrow 0^+} [\sin x \cdot x] &= \lim_{x \rightarrow 0} \sin x \cdot \lim_{x \rightarrow 0} x \\ &= 0 \cdot 0 \\ &= 0\end{aligned}$$

- **Propriedade do quociente** (desde que o denominador não tenda a zero), conforme a Equação 40:

$$\lim_{x \rightarrow a} \left[\frac{f(x)}{g(x)} \right] = \frac{\lim_{x \rightarrow a} f(x)}{\lim_{x \rightarrow a} g(x)} \quad (40)$$

Calcule os seguintes limites, aplicando a propriedade do quociente:

a) $\lim_{x \rightarrow 2} \left[\frac{x^2 - 1}{x - 1} \right]$

b) $\lim_{x \rightarrow 1} \left[\frac{x^2 + 4x + 4}{x + 2} \right]$

c) $\lim_{x \rightarrow 1} \left[\frac{\ln x}{x} \right]$

Soluções:

a)

$$\begin{aligned}\lim_{x \rightarrow 2} \left[\frac{x^2 - 1}{x - 1} \right] &= \frac{\lim_{x \rightarrow 2} (x^2 - 1)}{\lim_{x \rightarrow 2} (x - 1)} \\ &= \frac{3}{1} \\ &= 3\end{aligned}$$

b)

$$\begin{aligned}\lim_{x \rightarrow 1} \left[\frac{x^2 + 4x + 4}{x + 2} \right] &= \frac{\lim_{x \rightarrow 1} (x^2 + 4x + 4)}{\lim_{x \rightarrow 1} (x + 2)} \\ &= \frac{9}{3} \\ &= 3\end{aligned}$$

c)

$$\begin{aligned}\lim_{x \rightarrow 1} \left[\frac{\ln x}{x} \right] &= \frac{\lim_{x \rightarrow 1} \ln x}{\lim_{x \rightarrow 1} x} \\ &= \frac{0}{1} \\ &= 0\end{aligned}$$

Essas propriedades dos limites não apenas simplificam o cálculo de funções compostas, como também constituem ferramentas essenciais na análise de funções em diversas áreas do conhecimento, tais como física, economia e engenharia. Ao dominar essas propriedades, o estudante é capaz de resolver problemas com maior eficiência e precisão, especialmente em contextos que exigem modelagem matemática de fenômenos reais.

4.2.3 SCRIPT PYTHON DE OPERAÇÕES BÁSICAS COM LIMITES

Os *scripts* em Python podem ser construídos em dois estilos distintos de programação: o modo procedural e o modo orientado a objetos (POO). No modo procedural, o código é estruturado em funções independentes que executam tarefas específicas, utilizando variáveis globais ou passadas como parâmetros. Esse estilo é direto e simples, ideal para scripts pequenos e para quem está começando a programar. Ele favorece a clareza e a rapidez na implementação, mas pode tornar-se difícil de manter e expandir em projetos maiores, pois não organiza os dados e comportamentos em estruturas associadas. Já no modo orientado a objetos, o código é organizado em classes que agrupam dados (atributos) e funções (métodos) relacionadas. Isso permite criar objetos que representam conceitos do problema, facilitando a reutilização, manutenção e escalabilidade do código. No exemplo apresentado, criamos uma classe para encapsular as operações com limites, deixando o código mais modular e fácil de estender. O POO é especialmente recomendado para projetos complexos, onde a organização e a hierarquia do código são essenciais.

Em resumo, a escolha entre programação procedural e orientada a objetos depende do contexto do projeto, da experiência do programador e dos requisitos de manutenção e expansão do sistema.

Script Python em modo procedural

```
# Propriedade: Soma
from sympy import symbols, limit
x = symbols('x')
f = x**2
g = 3*x
limite_soma = limit(f + g, x, 2)
print(limite_soma) # Saída: 10

# Propriedade: Produto
f = x + 1
g = 2*x
limite_produto = limit(f * g, x, 3)
print(limite_produto) # Saída: 24

# Propriedade: Quociente
f = x**2 - 1
```

```

g = x - 1
limite_quociente = limit(f / g, x, 2)
print(limite_quociente) # Saída: 3

```

Observe que, nos exemplos resolvidos acima, apenas o item a de cada propriedade foi desenvolvido detalhadamente. Com base na estrutura apresentada e aplicando os mesmos princípios, recomenda-se que o leitor resolva também os itens b e c de cada caso, a fim de consolidar o aprendizado e exercitar a aplicação das propriedades estudadas.

Script Python em modo POO simplificado

```

from sympy import symbols, limit

class Limite:
    def __init__(self):
        self.x = symbols('x')

    def soma(self, f, g, a):
        return limit(f + g, self.x, a)

    def produto(self, f, g, a):
        return limit(f * g, self.x, a)

    def quociente(self, f, g, a):
        return limit(f / g, self.x, a)

# Utilizando a classe Limite
calc = Limite()
x = calc.x # variavel simbolica
# Limite da soma: f(x) = x^2, g(x) = 3x, x -> 2
print("Soma:", calc.soma(x**2, 3*x, 2)) # Saída: 10
# Limite do produto: f(x) = x+1, g(x) = 2x, x -> 3
print("Produto:", calc.produto(x + 1, 2*x, 3)) # Saída: 24
# Limite do quociente: f(x) = x^2 - 1, g(x) = x - 1, x -> 2
print("Quociente:", calc.quociente(x**2 - 1, x - 1, 2)) # Saída: 3

```

4.3 DERIVADAS

A **derivada** de uma função $f(x)$ é uma medida quantitativa de como a função varia em relação à sua variável independente x . Intuitivamente, a derivada expressa a inclinação da reta tangente à curva da função em um ponto específico, indicando a taxa de variação instantânea da função nesse ponto. A definição formal de uma derivada é dada pelo limite do quociente de diferença, conforme expressa na Equação 41:

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} \quad (41)$$

Este limite representa a inclinação da reta tangente à curva de $f(x)$ em um ponto. Se a derivada existir para todos os pontos de $f(x)$, dizemos que a *função é diferenciável nesse intervalo*. As derivadas

são fundamentais para a modelagem de processos dinâmicos, como velocidade e aceleração em sistemas físicos.

4.3.1 PROPRIEDADES DAS DERIVADAS

As derivadas seguem diversas propriedades fundamentais, que permitem o cálculo eficiente de derivadas de funções compostas ou expressões algébricas envolvendo múltiplas funções elementares.

- **Linearidade:** A derivada de uma soma ponderada de funções é igual à soma ponderada das derivadas dessas funções. Se $f(x)$ e $g(x)$ são diferenciáveis e a e b são constantes, então:

$$\frac{d}{dx}[af(x) + bg(x)] = af'(x) + bg'(x) \quad (42)$$

Calcule as seguintes derivadas utilizando a propriedade de linearidade:

- a) $\frac{d}{dx}[2x^2 + 5(3x)]$
- b) $\frac{d}{dx}[4\ln(x) - 3x^3]$
- c) $\frac{d}{dx}[7e^x + \frac{1}{2}x]$

Solução do item (a):

$$\begin{aligned} \frac{d}{dx}[2x^2 + 5(3x)] &= 2 \cdot \frac{d}{dx}[x^2] + 5 \cdot \frac{d}{dx}[3x] \\ &= 2 \cdot 2x + 5 \cdot 3 \\ &= 4x + 15 \end{aligned}$$

- **Regra do Produto:** A derivada do produto de duas funções diferenciáveis é dada por:

$$\frac{d}{dx}[f(x)g(x)] = f'(x)g(x) + f(x)g'(x) \quad (43)$$

Calcule as seguintes derivadas utilizando a regra do produto:

- a) $\frac{d}{dx}[x^2 \cdot \sin(x)]$
- b) $\frac{d}{dx}[x \cdot \ln(x)]$
- c) $\frac{d}{dx}[e^x \cdot \cos(x)]$

Solução do item (a):

$$\begin{aligned} \frac{d}{dx}[x^2 \cdot \sin(x)] &= \frac{d}{dx}[x^2] \cdot \sin(x) + x^2 \cdot \frac{d}{dx}[\sin(x)] \\ &= 2x \cdot \sin(x) + x^2 \cdot \cos(x) \end{aligned}$$

- **Regra da Cadeia:** A derivada da composição de funções $f(g(x))$ é obtida pela multiplicação da derivada externa com a derivada interna:

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x) \quad (44)$$

Calcule as seguintes derivadas utilizando a regra da cadeia:

- a) $\frac{d}{dx}[\cos^3(x)]$
- b) $\frac{d}{dx}[\ln(x^2 + 1)]$
- c) $\frac{d}{dx}[\sqrt{e^x}]$

Solução do item (a):

$$\begin{aligned} \frac{d}{dx}[\cos^3(x)] &= \frac{d}{dx}[(\cos x)^3] \\ &= 3\cos^2(x) \cdot \frac{d}{dx}[\cos x] \\ &= 3\cos^2(x) \cdot (-\sin x) \\ &= -3\cos^2(x)\sin(x) \end{aligned}$$

Essas propriedades das derivadas são ferramentas fundamentais para o cálculo simbólico e a análise de funções compostas em problemas aplicados.

4.3.2 SCRIPT PYTHON DE OPERAÇÕES BÁSICAS COM DERIVADAS

Script Python em modo procedural

```
import sympy as sp
# Exemplo: Linearidade
x = sp.symbols('x')
f = 2*x**2 + 15*x
derivative = sp.diff(f, x)
print(derivative) # Saída: 4*x + 15

# Exemplo: Regra do produto
f = x**2
g = sp.sin(x)
product_derivative = sp.diff(f * g, x)
print(product_derivative) # Saída: 2*x*sin(x) + x**2*cos(x)

# Exemplo: Regra da cadeia
u = sp.cos(x)
f_u = u**3
chain_derivative = sp.diff(f_u, x)
print(chain_derivative) # Saída: -3*cos(x)**2*sin(x)
```

Note que apenas o item *a* de cada propriedade de derivadas foi resolvido detalhadamente; recomenda-se ao leitor que resolva os itens *b* e *c*, aplicando os mesmos princípios, a fim de consolidar o aprendizado por meio da prática.

Script Python em modo POO simplificado

```
import sympy as sp
class DerivadaCalculator:
    def __init__(self):
        self.x = sp.symbols('x')

    def derivada_linearidade(self, expr):
        """
        Calcula a derivada da soma ponderada de funcoes,
        representada pela expressao simbolica expr.
        """
        return sp.diff(expr, self.x)

    def derivada_produto(self, f, g):
        """
        Calcula a derivada do produto f(x)*g(x) usando a regra do produto.
        """
        produto = f * g
        return sp.diff(produto, self.x)

    def derivada_cadeia(self, f_u):
        """
        Calcula a derivada da funcao composta f(u(x)) usando a regra da cadeia,
        em que f_u eh a expressao simbolica da funcao composta.
        """
        return sp.diff(f_u, self.x)

# Utilizando a classe DerivadaCalculator
calc = DerivadaCalculator()
x = calc.x
# Linearidade
f_linear = 2*x**2 + 15*x
print("Derivada (linearidade):", calc.derivada_linearidade(f_linear)) # Saida: 4*x + 15
# Regra do produto
f_prod = x**2
g_prod = sp.sin(x)
print("Derivada (produto):", calc.derivada_produto(f_prod, g_prod)) # Saida: 2*x*sin(x) + x**2*cos(x)
# Regra da cadeia
u = sp.cos(x)
f_u = u**3
print("Derivada (cadeia):", calc.derivada_cadeia(f_u)) # Saida: -3*cos(x)**2*sin(x)
```

4.4 INTEGRAIS

A **integração**, ou cálculo integral, é o processo inverso da diferenciação. Ela é usada para acumular valores ao longo de um intervalo, sendo fundamental para problemas que envolvem áreas sob curvas,

volumes de sólidos de revolução e até mesmo em equações diferenciais. A integral de uma função $f(x)$ no intervalo $[a, b]$ é definida como na Equação 45:

$$\int_a^b f(x) dx \quad (45)$$

A integral definida acumula a área sob a curva de $f(x)$ entre $x = a$ e $x = b$, enquanto a integral indefinida está associada à anti-derivada de $f(x)$. Integrais aparecem em muitos contextos, desde o cálculo de áreas e volumes até a resolução de equações diferenciais que descrevem sistemas dinâmicos.

4.4.1 PROPRIEDADES DAS INTEGRAIS

Assim como as derivadas, as integrais possuem propriedades que facilitam o cálculo, especialmente no caso de funções compostas ou definidas por pedaços. A seguir, apresentamos algumas propriedades fundamentais com exemplos ilustrativos.

- **Linearidade:** A integral de uma combinação linear de funções é igual à combinação linear das integrais dessas funções. Para funções $f(x)$, $g(x)$ e constantes a , b , temos:

$$\int_{x_0}^{x_1} [af(x) + bg(x)] dx = a \int_{x_0}^{x_1} f(x) dx + b \int_{x_0}^{x_1} g(x) dx \quad (46)$$

Calcule as seguintes integrais definidas, aplicando a propriedade da linearidade:

- a) $\int_0^1 [3x^2 + 2x] dx$
- b) $\int_1^2 [4 \ln x - 5x] dx$
- c) $\int_0^2 [x + 6e^x] dx$

Solução do item (a):

$$\begin{aligned} \int_0^1 [3x^2 + 2x] dx &= 3 \int_0^1 x^2 dx + 2 \int_0^1 x dx \\ &= 3 \left[\frac{x^3}{3} \right]_0^1 + 2 \left[\frac{x^2}{2} \right]_0^1 \\ &= 3 \cdot \frac{1}{3} + 2 \cdot \frac{1}{2} \\ &= 1 + 1 \\ &= 2 \end{aligned}$$

- **Adição de Intervalos:** A integral de uma função em um intervalo pode ser dividida em subintervalos. Para $a < c < b$, temos:

$$\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx \quad (47)$$

Calcule as seguintes integrais definidas utilizando a adição de intervalos:

$$\text{a)} \quad \int_0^2 x^2 dx = \int_0^1 x^2 dx + \int_1^2 x^2 dx$$

$$\text{b)} \quad \int_1^3 x dx = \int_1^2 x dx + \int_2^3 x dx$$

$$\text{c)} \quad \int_0^2 e^x dx = \int_0^1 e^x dx + \int_1^2 e^x dx$$

Solução do item (a):

$$\begin{aligned} \int_0^2 x^2 dx &= \int_0^1 x^2 dx + \int_1^2 x^2 dx \\ &= \left[\frac{x^3}{3} \right]_0^1 + \left[\frac{x^3}{3} \right]_1^2 \\ &= \frac{1^3}{3} - \frac{0^3}{3} + \left(\frac{8}{3} - \frac{1}{3} \right) \\ &= \frac{1}{3} + \frac{7}{3} \\ &= \frac{8}{3} \end{aligned}$$

As integrais desempenham um papel crucial na modelagem de fenômenos acumulativos, como o cálculo de áreas, volumes, trabalho em sistemas físicos e até mesmo em análises estatísticas e econômicas.

4.4.2 SCRIPT PYTHON DE OPERAÇÕES BÁSICAS COM INTEGRAIS

Script Python em modo procedural

```
import sympy as sp

# Definindo a variavel simbolica
x = sp.symbols('x')

# Definindo as funcoes
f = 3*x**2 + 2*x
g = x**2

# Calculando a integral de 0 a 1 para 3f(x) + 2g(x)
integral_f_g = sp.integrate(f, (x, 0, 1))

# Calculando as integrais separadamente
integral_f = sp.integrate(g, (x, 0, 1))
integral_g = sp.integrate(x, (x, 0, 1))

# Resultados
result_linearidade = 3 * integral_f + 2 * integral_g

# Exibindo os resultados
print("Resultado da integral de 3x^2 + 2x de 0 a 1:", integral_f_g)
print("Resultado da integral de 3x^2 + 2x (usando linearidade):", result_linearidade
)

# Calculando a integral de x^2 de 0 a 2 usando adicao de intervalos
```



```

integral_0_1 = sp.integrate(g, (x, 0, 1))
integral_1_2 = sp.integrate(g, (x, 1, 2))

# Resultado final
result_adicao_intervalos = integral_0_1 + integral_1_2
print("Resultado da integral de x^2 de 0 a 2 usando adicao de intervalos:",
      result_adicao_intervalos)

```

Observe que, nos exemplos resolvidos acima, apenas o item *a* de cada propriedade foi desenvolvido detalhadamente. É altamente recomendável que o leitor resolva os itens *b* e *c* para fixar os conceitos e aplicar as propriedades com autonomia.

Script Python em modo POO simplificado

```

import sympy as sp
class IntegralCalculator:
    def __init__(self):
        # Define a variavel simbolica
        self.x = sp.symbols('x')

    def calcular_integral_linearidade(self, f, g, a, b, coef_f, coef_g):
        # Calcula a integral direta de coef_f*f + coef_g*g no intervalo [a, b]
        total_expr = coef_f * f + coef_g * g
        integral_total = sp.integrate(total_expr, (self.x, a, b))

        # Calcula as integrais separadas e aplica a propriedade da linearidade
        integral_f = sp.integrate(f, (self.x, a, b))
        integral_g = sp.integrate(g, (self.x, a, b))
        integral_linear = coef_f * integral_f + coef_g * integral_g

        return integral_total, integral_linear

    def calcular_integral_adicao_intervalos(self, expr, a, b, c):
        # Divide o intervalo [a, b] em [a, c] e [c, b] e soma as integrais
        integral_ac = sp.integrate(expr, (self.x, a, c))
        integral_cb = sp.integrate(expr, (self.x, c, b))
        soma = integral_ac + integral_cb
        return soma

# Instancia o objeto calculador
calc = IntegralCalculator()
x = calc.x # Variavel simbolica
# Define as expressoes simbolicas
f = 3*x**2 + 2*x
g = x**2
# Calcula a integral de 3x^2 + 2x no intervalo [0, 1]
integral_direta, integral_linear = calc.calcular_integral_linearidade(f=x**2, g=x, a=0, b=1, coef_f=3, coef_g=2)
# Exibe os resultados
print("Integral direta de 3x^2 + 2x no intervalo [0,1]:", integral_direta)
print("Integral usando propriedade da linearidade:", integral_linear)

```

```
# Calcula a integral de x^2 de 0 a 2 dividindo em 0-1 e 1-2
resultado_soma = calc.calcular_integral_adicao_intervalos(expr=x**2, a=0, b=2, c=1)
print("Integral de x^2 de 0 a 2 usando adicao de intervalos:", resultado_soma)
```

4.5 CONCLUSÃO

Com uma compreensão sólida dos conceitos matemáticos fundamentais — como limites, derivadas e integrais — torna-se mais confortável avançar no estudo de técnicas numéricas aplicadas à engenharia. Desse modo, ao longo dos capítulos seguintes, utilizaremos a linguagem **Python**; uma das mais poderosas e acessíveis para aplicações científicas e de engenharia, tanto no estilo de programação **procedural** quanto no paradigma **orientado a objetos (POO)**. Esses diferentes estilos de programação oferecem flexibilidade na implementação de algoritmos: o modo *procedural* é ideal para *scripts* diretos e rápidos, enquanto o modo *POO* favorece a modularidade, reutilização e expansão de código em projetos mais complexos. Ambos serão úteis para a construção de soluções robustas e didáticas no contexto de métodos numéricos.

Com essas ferramentas computacionais e conceituais, resolveremos problemas práticos como simulação de sistemas dinâmicos, otimização de processos e análise estrutural, integrando teoria e prática de forma eficiente. Adicionalmente, códigos complementares sobre cálculo simbólico e exemplos adicionais utilizando outras bibliotecas estão disponíveis no repositório: <https://github.com/jonathacosta/NM/tree/main/NM_codes/Un01>

LISTA DE EXERCÍCIOS - CÁLCULO: LIMITES, DERIVADAS E INTEGRAIS

PARTE 1: LIMITES

1. **Cálculo Direto de Limites** Calcule os limites abaixo:

a) $\lim_{x \rightarrow 3} \frac{x^2 - 9}{x - 3}$

b) $\lim_{x \rightarrow 0} \frac{\sin(5x)}{x}$

c) $\lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x$

d) $\lim_{x \rightarrow -2} \frac{\sqrt{4+x} - 2}{x+2}$

2. **Limites Laterais** Verifique se o limite existe e calcule-o:

a) $\lim_{x \rightarrow 0^+} \frac{1}{x}$

b) $\lim_{x \rightarrow 0^-} \frac{1}{x}$

c) Explique por que o limite $\lim_{x \rightarrow 0} \frac{1}{x}$ não existe com base nos limites laterais calculados.

3. **Limites e Funções Trigonômicas** Encontre o limite:

a) $\lim_{x \rightarrow 0} \frac{\tan(x) - \sin(x)}{x^3}$

b) $\lim_{x \rightarrow 0} \frac{\sin^2(x)}{x^2}$

PARTE 2: DERIVADAS

4. **Derivação Direta** Calcule a derivada das funções abaixo:

a) $f(x) = x^3 - 4x + 2$

b) $g(x) = \ln(x^2 + 1)$

c) $h(x) = e^{3x} \cdot \sin(x)$

5. **Derivadas de Ordem Superior** Determine a segunda e terceira derivada de $f(x) = x^4 - 3x^2 + x$.

PARTE 3: INTEGRAIS

9. **Integração Direta** Calcule as integrais indefinidas e defina as constantes de integração:

a) $\int (3x^2 - 4x + 1) dx$

b) $\int \frac{1}{x \ln(x)} dx$

c) $\int e^{2x} \sin(x) dx$

10. **Integrais Definidas** Calcule as integrais definidas abaixo:

a) $\int_0^1 (x^3 - 3x^2 + 2) dx$

b) $\int_1^e \frac{\ln(x)}{x} dx$

c) $\int_0^\pi x \sin(x) dx$

11. **Integrais ImproPRIas** Avalie, se convergente:

a) $\int_1^\infty \frac{1}{x^2} dx$

b) $\int_0^\infty e^{-x^2} dx$

c) $\int_0^\infty \frac{\sin(x)}{x} dx$

PARTE 4: EXERCÍCIOS DE APLICAÇÃO

12. **Modelagem Matemática com Derivadas e Integrais** Considere uma população de peixes em um lago modelada pela função $P(t) = 5000e^{0.1t}$, onde t é o tempo em anos.

- a) Encontre a taxa de crescimento da população no instante $t = 5$ anos.
- b) Determine o tempo necessário para que a população atinja 10.000 peixes.

5 Sistemas de Numeração

Os sistemas de numeração são a base da representação dos números e formam a estrutura essencial para a computação moderna. Diferentes sistemas de numeração foram desenvolvidos ao longo da história, cada um com suas próprias características e usos. Este capítulo discute os conceitos fundamentais dos sistemas de numeração, com foco em quatro sistemas amplamente utilizados: decimal, binário, octal e hexadecimal.

5.1 CONCEITUAÇÃO DE SISTEMAS DE NUMERAÇÃO

Um sistema de numeração é uma forma de representar números utilizando um conjunto finito de símbolos. Cada sistema é baseado em uma **base**, que determina o número de símbolos distintos que podem ser usados, e na posição dos dígitos, que corresponde a potências da base. Em geral, qualquer número em um sistema de base b pode ser representado da seguinte forma:

$$N = d_n \times b^n + d_{n-1} \times b^{n-1} + \dots + d_1 \times b^1 + d_0 \times b^0$$

Em que:

- b é a base do sistema de numeração (por exemplo, 2 no binário, 10 no decimal, etc.);
- d_i são os dígitos da representação numérica, com $0 \leq d_i < b$;
- n é o expoente máximo, que corresponde à posição do dígito mais à esquerda.

A base define o número de dígitos possíveis, e cada dígito multiplica uma potência de b , dependendo de sua posição. Quanto mais à esquerda o dígito está, maior a potência de b que o acompanha.

5.1.1 SISTEMA DECIMAL (BASE 10)

O sistema decimal é o mais familiar para os seres humanos, pois é o sistema que usamos no dia a dia. Ele utiliza dez dígitos (0 a 9) e a posição de cada dígito tem um valor baseado em potências de 10.

Por exemplo, o número 345 no sistema decimal é expresso como:

$$345 = 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$$

5.1.2 SISTEMA BINÁRIO (BASE 2)

O sistema binário é amplamente utilizado em computadores e sistemas digitais. Ele utiliza apenas dois dígitos, 0 e 1, e cada posição representa uma potência de 2.

Por exemplo, o número binário 1011 é expresso como:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}$$

5.1.3 SISTEMA OCTAL (BASE 8)

O sistema octal utiliza oito dígitos (0 a 7) e cada posição tem um valor baseado em potências de 8. Esse sistema é menos comum, mas ainda é utilizado em algumas aplicações digitais, especialmente na eletrônica.

Por exemplo, o número octal 73 é expresso como:

$$73_8 = 7 \times 8^1 + 3 \times 8^0 = 59_{10}$$

5.1.4 SISTEMA HEXADECIMAL (BASE 16)

O sistema hexadecimal utiliza 16 dígitos, que incluem os números de 0 a 9 e as letras A, B, C, D, E e F, representando os valores de 10 a 15. Esse sistema é muito utilizado em programação e eletrônica devido à sua compactação eficiente de números binários.

Por exemplo, o número hexadecimal 1A3F é expresso como:

$$1A3F_{16} = 1 \times 16^3 + A \times 16^2 + 3 \times 16^1 + F \times 16^0 = 1 \times 16^3 + 10 \times 16^2 + 3 \times 16^1 + 15 \times 16^0 = 6719_{10}$$

5.2 CONVERSÃO ENTRE SISTEMAS DE NUMERAÇÃO

5.2.1 CONVERSÃO DE DECIMAL PARA BINÁRIO

Para converter um número decimal para binário, utilizamos divisões sucessivas por 2 e anotamos os restos. Então, reconstruímos o número a partir do último quociente e em direção ao primeiro resto das sucessivas divisões.

Por exemplo, para converter o número decimal 13 para binário:

$$13 \div 2 = 6 \text{ resto } \mathbf{1}$$

$$6 \div 2 = 3 \text{ resto } \mathbf{0}$$

$$3 \div 2 = 1 \text{ resto } \mathbf{1}$$

$$1 \div 2 = \mathbf{0} \text{ resto } \mathbf{1}$$

O número 'n' em binário é, portanto, '**qrrrr**'. Desse modo, o número 13 em binário é 01101.

5.2.2 CONVERSÃO DE BINÁRIO PARA DECIMAL

Para converter um número binário para decimal, multiplicamos cada dígito por sua respectiva potência de 2 e somamos os resultados.

Por exemplo, para converter o número binário 1101 para decimal:

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

5.2.3 CONVERSÃO DE DECIMAL PARA HEXADECIMAL

A conversão de decimal para hexadecimal segue um processo similar ao de decimal para binário, mas usamos divisões sucessivas por 16.

Por exemplo, para converter o número decimal 6719 para hexadecimal:

$$6719 \div 16 = 419 \text{ resto } \mathbf{15} (F)$$

$$419 \div 16 = 26 \text{ resto } \mathbf{3} (3)$$

$$26 \div 16 = 1 \text{ resto } \mathbf{10} (A)$$

$$1 \div 16 = 0 \text{ resto } \mathbf{1} (1)$$

Assim, o número 6719 em hexadecimal é 1A3F.

5.2.4 CONVERSÃO DE HEXADECIMAL PARA DECIMAL

A conversão de hexadecimal para decimal segue o processo de multiplicar cada dígito pelo valor correspondente da potência de 16.

Por exemplo, para converter o número hexadecimal 1A3F para decimal:

$$1A3F_{16} = 1 \times 16^3 + A \times 16^2 + 3 \times 16^1 + F \times 16^0 = 6719_{10}$$

5.2.5 PASSOS PARA A CONVERSÃO DE UM NÚMERO FRACIONÁRIO DE BASE B PARA DECIMAL

- Para cada dígito à direita da vírgula, multiplicamos o dígito por b^{-n} , onde b é a base do sistema e n é a posição do dígito. A 1ª posição após a vírgula corresponde a b^{-1} , a 2ª posição a b^{-2} , e assim por diante.
- Somamos todos os valores obtidos para chegar ao resultado final em decimal.

Exemplo Genérico: Converter $0.d_1d_2d_3 \dots_b$ para decimal.

$$0.d_1d_2d_3 \dots_b = d_1 \times b^{-1} + d_2 \times b^{-2} + d_3 \times b^{-3} + \dots$$

Exemplo Específico em Base 3: Converter 0.2103_3 para decimal.

$$0.2103_3 = 2 \times 3^{-1} + 1 \times 3^{-2} + 0 \times 3^{-3} + 3 \times 3^{-4}$$

$$0.2103_3 = \frac{2}{3} + \frac{1}{9} + 0 + \frac{3}{81}$$

$$0.2103_3 = 0.6667 + 0.1111 + 0 + 0.0370 = 0.8148_{10}$$

Esses passos podem ser seguidos para converter qualquer número fracionário de uma base b para o sistema decimal.

5.3 CONVERSÃO DE SISTEMAS COM VÍRGULA

A conversão de números compostos por uma parte inteira e uma parte fracionária é realizada separadamente para cada parte. Primeiramente, a parte inteira é convertida utilizando o processo de conversão para números inteiros descrito anteriormente. Em seguida, a parte fracionária é convertida de acordo com o método específico para frações. Após a conversão de ambas as partes, os resultados são combinados para formar o número completo no sistema de destino.

Matematicamente, isso pode ser expresso da seguinte forma:

$$\text{número convertido} = \text{parte inteira convertida} + \text{parte fracionária convertida}$$

Dessa forma, o número final no sistema de destino será a justaposição da parte inteira e da parte fracionária já convertidas.

5.3.1 PASSOS PARA A CONVERSÃO FRACIONÁRIO PARA DECIMAL

A conversão de números fracionários (com vírgula) segue um processo semelhante ao utilizado para números inteiros contundo, ao invés de utilizar as potências inteiras da base do sistema, utilizamos potências negativas dessa base para os dígitos à direita da vírgula.

- Para cada dígito à direita da vírgula, multiplicamos o dígito por $base^{-n}$, onde n é a posição do dígito (1ª posição após a vírgula corresponde a $base^{-1}$, 2ª posição corresponde a $base^{-2}$, e assim por diante).
- Somamos todos os valores obtidos para chegar ao resultado final em decimal.

5.3.2 APLICAÇÃO DO PROCEDIMENTO DE CONVERSÃO

Considere, portanto, os exemplos a seguir e o procedimento acima descrito:

a) Converter 0.1101_2 para decimal.

Parte inteira: 0_2

Parte fracionária: 1101_2

Conversão:

$$\begin{aligned} 0.1101_2 &= 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} \\ 0.1101_2 &= \frac{1}{2} + \frac{1}{4} + 0 + \frac{1}{16} \\ 0.1101_2 &= 0.5_{10} + 0.25_{10} + 0_{10} + 0.0625_{10} = 0.8125_{10} \end{aligned}$$

Resultado: $0.1101_2 = 0.8125_{10}$.

b) Converter 11.1101_2 para decimal.

Parte inteira: 11_2

Parte fracionária: 1101_2

Conversão:

Parte inteira: 11_2

$$11_2 = 1 \times 2^1 + 1 \times 2^0$$

$$11_2 = 2_{10} + 1_{10} = 3_{10}$$

Parte fracionária: 0.1101_2

$$1101_2 = 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$

$$1101_2 = \frac{1}{2} + \frac{1}{4} + 0 + \frac{1}{16}$$

$$1101_2 = 0.5_{10} + 0.25_{10} + 0_{10} + 0.0625_{10} = 0.8125_{10}$$

Sendo: $11_2 = 3_{10}$ e $0.1101_2 = 0.8125_{10}$, tem-se que o resultado de $11.1101_2 = 3.8125_{10}$.

c) Converter $0.1A3_{16}$ para decimal.

Parte inteira: 0_{16}

Parte fracionária: $1A3_{16}$

Conversão:

$$0.1A3_{16} = 1 \times 16^{-1} + A \times 16^{-2} + 3 \times 16^{-3}$$

$$0.1A3_{16} = 1 \times \frac{1}{16} + 10 \times \frac{1}{16^2} + 3 \times \frac{1}{16^3}$$

$$0.1A3_{16} = 0.0625 + 10 \times 0.00390625 + 3 \times 0.00024414$$

$$0.1A3_{16} = 0.0625 + 0.0390625 + 0.00073242 = 0.10229492_{10}$$

Resultado: $0.1A3_{16} = 0.10229492_{10}$.

d) Converter $C.1A3_{16}$ para decimal.

Parte inteira: C_{16}

Parte fracionária: $1A3_{16}$

Conversão:

Parte inteira: C_{16}

$$C_{16} = 12_{10}$$

Note que, no caso do sistema hexadecimal, os dígitos A, B, C, D, E, F no hexadecimal correspondem aos valores 10, 11, 12, 13, 14, 15 em decimal.

$$0.1A3_{16} = 1 \times 16^{-1} + A \times 16^{-2} + 3 \times 16^{-3}$$

$$0.1A3_{16} = 1 \times \frac{1}{16} + 10 \times \frac{1}{16^2} + 3 \times \frac{1}{16^3}$$

$$0.1A3_{16} = 0.0625 + 10 \times 0.00390625 + 3 \times 0.00024414$$

$$0.1A3_{16} = 0.0625_{10} + 0.0390625_{10} + 0.00073242_{10} = 0.10229492_{10}$$

Sendo: $C_{16} = 12_{10}$ e $1A3_{16} = 0.10229492_{10}$, tem-se que o resultado de $C.1A3_{16} = 12.10229492_{10}$.

5.4 SCRIPT PYTHON PARA CONVERSÃO ENTRE SISTEMAS

O código a seguir ilustra uma forma simplificada e compacta de conversão entre sistemas de numeração no Python, mantendo a funcionalidade de conversão de números inteiros e fracionários.

```
def decimal_para_base(n, base):
    parte_inteira = int(n)
    parte_fracionaria = n - parte_inteira

    # Conversao da parte inteira
    if base == 2:
        inteiro_convertido = bin(parte_inteira)[2:]
    elif base == 8:
        inteiro_convertido = oct(parte_inteira)[2:]
    elif base == 16:
        inteiro_convertido = hex(parte_inteira)[2:].upper()
    else:
        raise ValueError("Base nao suportada. Escolha 2 (binario), 8 (octal) ou 16 (hexadecimal).")

    # Conversao da parte fracionaria
    fracao_convertida = ""
    while parte_fracionaria > 0 and len(fracao_convertida) < 10: # Limite de precisao
        parte_fracionaria *= base
        digito = int(parte_fracionaria)
        fracao_convertida += (hex(digito)[2:].upper() if base == 16 else str(digito))
        parte_fracionaria -= digito

    return inteiro_convertido + ('.' + fracao_convertida if fracao_convertida else "")

def base_para_decimal(num_str, base):
    if '.' in num_str:
        parte_inteira, parte_fracionaria = num_str.split('.')
    else:
        parte_inteira, parte_fracionaria = num_str, ''

    # Conversao da parte inteira
    dec_inteiro = int(parte_inteira, base)

    # Conversao da parte fracionaria
    dec_fracao = sum(int(digito, base) * (base ** -(i + 1)) for i, digito in enumerate(parte_fracionaria))

    return dec_inteiro + dec_fracao

# Testes
numero_decimal = 13.8125
print(f"Decimal {numero_decimal} para binario: {decimal_para_base(numero_decimal, 2)}")
print(f"Decimal {numero_decimal} para hexadecimal: {decimal_para_base(numero_decimal, 16)}")
```

```

, 16))")
print(f"Decimal {numero_decimal} para octal: {decimal_para_base(numero_decimal, 8)}"
)

numero_binario = "1101.1101"
print(f"Binario {numero_binario} para decimal: {base_para_decimal(numero_binario, 2)
}")

numero_hexadecimal = "C.1A3"
print(f"Hexadecimal {numero_hexadecimal} para decimal: {base_para_decimal(
    numero_hexadecimal, 16)}")

numero_octal = "15.63"
print(f"Octal {numero_octal} para decimal: {base_para_decimal(numero_octal, 8)}")

```

Respostas do *script*:

Decimal 13.8125 para binário: 1101.1101

Decimal 13.8125 para hexadecimal: D.D

Decimal 13.8125 para octal: 15.64

Binário 1101.1101 para decimal: 13.8125

Hexadecimal C.1A3 para decimal: 12.102294921875

Octal 15.63 para decimal: 13.796875

5.5 CONCLUSÃO

Compreender os sistemas de numeração e as técnicas de conversão entre eles é fundamental para o trabalho com computadores e algoritmos numéricos. Estes conceitos formam a base para uma compreensão mais profunda dos métodos numéricos abordados nos capítulos subsequentes deste livro.

LISTA DE EXERCÍCIOS: SISTEMAS DE NUMERAÇÃO**1. Conversão de Bases**

- a) Converta o número 1011_2 (base 2) para a base 10.
- b) Converta o número $A3F_{16}$ (base 16) para a base 10.
- c) Converta o número 345_8 (base 8) para a base 10.

2. Conversão para Diferentes Bases

- a) Converta o número 205 da base 10 para a base 2.
- b) Converta o número 147 da base 10 para a base 8.
- c) Converta o número 255 da base 10 para a base 16.

3. Operações em Bases Diferentes

- a) Calcule $1101_2 + 101_2$ e apresente o resultado em base 2.
- b) Calcule $345_8 - 127_8$ e apresente o resultado em base 8.
- c) Calcule $A3_{16} + 5F_{16}$ e apresente o resultado em base 16.

4. Multiplicação e Divisão em Bases Diferentes

- a) Multiplique $1011_2 \times 110_2$ e apresente o resultado em base 2.
- b) Divida 725_8 por 3_8 e apresente o resultado em base 8.
- c) Multiplique $B2_{16} \times 3_{16}$ e apresente o resultado em base 16.

5. Complemento de 1 e Complemento de 2 em Base 2

- a) Encontre o complemento de 1 para o número binário 101010.
- b) Encontre o complemento de 2 para o número binário 100110.

6. Conversão com Ponto Decimal

- a) Converta o número decimal 10.75_{10} para a base 2.
- b) Converta o número decimal 23.25_{10} para a base 8.
- c) Converta o número decimal 125.5_{10} para a base 16.

6 Aritmética de Ponto Flutuante e Erros Computacionais

Neste capítulo, discutiremos a aritmética de ponto flutuante utilizada em computadores, como os dados numéricos são armazenados, e os diferentes tipos de erros que podem ocorrer em cálculos numéricos, como erros de arredondamento e de truncamento.

6.1 ARITMÉTICA DE PONTO FLUTUANTE

A aritmética de ponto flutuante é amplamente usada em cálculos numéricos, pois permite representar uma ampla gama de números, desde números muito pequenos até números muito grandes. Em geral, um número de ponto flutuante x pode ser representado como:

$$x = \pm m \times 2^e$$

em que m é a mantissa (ou significando), e é o expoente, e a base 2 é utilizada na maioria das representações de ponto flutuante em computadores modernos.

Os sistemas mais comuns de representação de ponto flutuante seguem um padrão normativo. A Tabela 2 mostra a distribuição dos bits na representação IEEE 754/2008 para números de precisão simples (32 bits) e dupla (64 bits).

Tabela 2 – Distribuição dos bits na representação IEEE 754/2008

Tipo	Simples (32 bits)	Dupla (64 bits)
Sinal	bit 31	bit 63
Expoente	bits 30-23	bits 62-52
Mantissa	bits 22-0	bits 51-0

Fonte: Autor, 2024

6.2 PASSO-A-PASSO GENERALIZADO PARA CONVERSÃO

O padrão IEEE 754 utiliza uma notação de ponto flutuante para representar números em formato binário. No processo genérico para converter um número decimal, como 81, para as representações de precisão simples (32 bits) e dupla (64 bits) devem seguir o seguinte passo-a-passo:

PASSO 1: REPRESENTAÇÃO BINÁRIA DO NÚMERO

Primeiro, converta o número inteiro decimal para binário. O número 81 em binário é:

$$81_{10} = 1010001_2$$

PASSO 2: REPRESENTAÇÃO NORMALIZADA

No formato IEEE 754, a mantissa é representada em notação normalizada. A notação normalizada desloca o ponto binário de modo que haja exatamente um dígito 1 à esquerda do ponto.

Para o número 81, tem-se:

$$1010001_2 = 1.010001 \times 2^6$$

Aqui, o número foi deslocado 6 posições para a esquerda, então o expoente é 6.

PASSO 3: CÁLCULO DO EXPOENTE COM VIÉS

No formato IEEE 754, o expoente armazenado não é o expoente real, mas sim um expoente com viés (bias). Para precisão simples (32 bits), o viés é 127, e para precisão dupla (64 bits), o viés é 1023.

- Para precisão simples:

$$\text{expoente armazenado} = 6 + 127 = 133$$

- Para precisão dupla:

$$\text{expoente armazenado} = 6 + 1023 = 1029$$

Os expoentes armazenados são 133 em decimal (ou 10000101_2 em binário) para precisão simples, e 1029 em decimal (ou 10000000101_2 em binário) para precisão dupla.

PASSO 4: MONTAGEM DOS BITS

Agora, montamos o número com base nos componentes:

- O bit de sinal é 0, pois 81 é positivo.
- O expoente é armazenado em 8 bits para precisão simples e 11 bits para precisão dupla.
- A mantissa é representada pelos dígitos após o ponto binário.

Representação em Precisão Simples (32 bits)

Montando a representação de precisão simples (32 bits):

- **Sinal:** 0
- **Expoente:** 10000101_2
- **Mantissa:** 0100010000000000000000_2

Logo, o número 81 em precisão simples é:

0 10000101 010001000000000000000000

Note que o valor 81, na Figura 7, é utilizado apenas como exemplo no processo. No entanto, o fluxo segue de forma semelhante para qualquer outro número, respeitando as devidas diferenças nas magnitudes dos valores e correspondências para binário.

6.4 SCRIPT EM PYTHON DE CONVERSÃO PARA PRECISÃO SIMPLES E DUPLA

```
def float_to_binary(num, precision):
    """Converte um numero decimal para ponto flutuante binario."""
    if num < 0:
        sign = 1
        num = -num
    else:
        sign = 0
    # Encontrar o expoente e a mantissa
    exponent = 0
    while num >= 2:
        num /= 2
        exponent += 1
    while num < 1:
        num *= 2
        exponent -= 1
    # Calcular o expoente com vies
    if precision == 32: # Precisao simples
        bias = 127
    elif precision == 64: # Precisao dupla
        bias = 1023
    else:
        raise ValueError("Precisao deve ser 32 (simples) ou 64 (dupla).")

    exponent += bias
    # Converter expoente para binario
    exponent_bits = bin(exponent)[2:] # Remove o prefixo '0b'
    exponent_bits = exponent_bits.zfill(8 if precision == 32 else 11) #
    # Preenche com zeros a esquerda
    # Calcular a mantissa
    mantissa = num - 1 # Remove o 1 a esquerda do ponto binario
    mantissa_bits = ""
    for _ in range(23 if precision == 32 else 52): # 23 bits para precisao
        # simples, 52 para dupla
        mantissa *= 2
        if mantissa >= 1:
            mantissa_bits += '1'
            mantissa -= 1
        else:
            mantissa_bits += '0'
    # Montar a representacao final
    if precision == 32:
        return f"{sign}{exponent_bits}{mantissa_bits}"
    else:
        return f"{sign}{exponent_bits}{mantissa_bits}"
```



```
# Testando a funcao
number = 81
simple_precision = float_to_binary(number, 32)
double_precision = float_to_binary(number, 64)

print(f"Numero: {number}")
print(f"Precisao Simples (32 bits): {simple_precision}")
print(f"Precisao Dupla (64 bits): {double_precision}")
```

6.5 ARMAZENAMENTO DE DADOS NO COMPUTADOR

Os computadores usam sistemas de numeração binários para armazenar dados. Números inteiros são armazenados diretamente em formato binário, enquanto números fracionários e irracionais requerem representações mais complexas, como a de ponto flutuante. Os números armazenados em computadores estão sujeitos a limitações de espaço (bits) e, como resultado, a precisão dos números armazenados é limitada. Esse fato tem implicações em cálculos numéricos, como veremos nas seções sobre erros de arredondamento e truncamento.

6.5.1 LIMITAÇÕES NA REPRESENTAÇÃO

Devido à limitação no número de bits, não é possível armazenar com exatidão todos os números reais. Por exemplo, números irracionais como π e e precisam ser aproximados, assim como frações que não possuem uma representação finita em binário, como $\frac{1}{3}$.

6.6 ERROS DE ARREDONDAMENTO E DE TRUNCAMENTO

Os erros computacionais são inevitáveis devido às limitações da aritmética de ponto flutuante. Dois tipos principais de erros podem ocorrer: erros de arredondamento e erros de truncamento.

6.6.1 ERROS DE ARREDONDAMENTO

Erros de arredondamento ocorrem quando um número não pode ser representado exatamente com o número limitado de bits disponíveis. Por exemplo, ao armazenar o número $\frac{1}{3}$, o computador deve arredondar sua representação para caber em uma quantidade finita de bits. Isso leva a pequenas discrepâncias entre o valor armazenado e o valor exato.

Matematicamente, podemos definir o erro de arredondamento E_r como expresso na Equação 48:

$$E_r = x - \hat{x} \quad (48)$$

em que x é o valor real e $\hat{x}$ é o valor armazenado no computador.

6.6.2 ERROS DE TRUNCAMENTO

Erros de truncamento ocorrem quando um processo de aproximação é interrompido ou truncado. Esses erros são comuns em séries numéricas e métodos iterativos que precisam ser interrompidos após um número finito de passos. Um exemplo típico é a aproximação de derivadas numéricas por diferenças finitas, em que a precisão depende do número de termos utilizados na aproximação.

O erro de truncamento E_t pode ser expresso como na Equação 49:

$$E_t = f(x) - P_n(x) \quad (49)$$

em que $f(x)$ é a função real e $P_n(x)$ é a aproximação da função após n termos.

6.7 CONCLUSÃO

Neste capítulo, exploramos a aritmética de ponto flutuante, a forma como os dados são armazenados em computadores e os erros que surgem em cálculos numéricos. O entendimento desses conceitos é fundamental para a aplicação correta de métodos numéricos, pois eles ajudam a identificar e mitigar erros em cálculos complexos.

7 Derivação Numérica

A derivação numérica é ferramenta fundamental no campo dos métodos numéricos. Elas permitem a aproximação de derivadas de funções que não podem ser resolvidas analiticamente, ou cuja a solução analítica é demasiadamente trabalhosa. Neste capítulo, abordaremos os principais métodos de derivação numérica, com ênfase em suas aplicações e limitações.

7.1 CONCEITUAÇÃO DE DERIVAÇÃO NUMÉRICA

A derivada de uma função $f(x)$ pode ser aproximada numericamente utilizando métodos baseados em diferenças finitas, uma técnica muito útil, especialmente em situações em que a diferenciação analítica, definida na Seção 4.3 é inviável ou excessivamente complexa. A derivação numérica é amplamente empregada para identificar os pontos de máximo e mínimo de uma função e se torna uma ferramenta eficaz quando a função é definida por valores discretos obtidos a partir de dados experimentais ou de medições.

A aproximação da derivada utilizando diferenças finitas baseia-se nos valores da função em diferentes pontos ao redor de $x = a$ para estimar a inclinação da curva. Existem três fórmulas básicas para derivadas finitas, que são as aproximações mais simples para esse tipo de cálculo:

- **Derivação progressiva:** utiliza os valores da função de x_i para x_{i+1} .
- **Derivação regressiva:** emprega os valores de x_i e x_{i-1} .
- **Derivação central:** faz uso dos valores entre x_{i-1} e x_{i+1} .

Esses métodos são fundamentais na derivação numérica e permitem uma estimativa precisa da inclinação da função em cada ponto, mesmo em casos em que a função só está definida em pontos discretos. Nesses métodos, a derivada no ponto (x_i) é calculada a partir do valor de dois pontos. A derivada é estimada como a inclinação da reta que conecta esses dois pontos, conforme também enfatiza (GILAT; SUBRAMANIAM, 2008).

7.2 DERIVAÇÃO PROGRESSIVA

A diferença progressiva é expressa na Equação 50:

$$f'(x) = \left. \frac{df(x)}{dx} \right|_{x=x_i} \approx \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} \quad (50)$$

A diferença progressiva (ou para frente) é uma aproximação numérica da derivada de uma função em um ponto x_i . Essa técnica estima a inclinação da tangente à curva da função $f(x)$ em x_i com base nos valores da função em x_i e em um ponto seguinte x_{i+1} . A fórmula apresentada na Equação 50 fornece essa aproximação, em que a derivada $f'(x)$ é aproximada pela razão entre a diferença nos valores da função $f(x_{i+1})$ e $f(x_i)$ e o espaçamento entre os pontos x_{i+1} e x_i .

7.3 DERIVAÇÃO REGRESSIVA

A diferença regressiva (ou para trás) é uma técnica de diferenciação numérica utilizada para aproximar a derivada de uma função em um determinado ponto. Neste método, a derivada da função $f(x)$ no ponto x_i é aproximada utilizando o valor da função no ponto x_{i-1} , que é o ponto imediatamente anterior a x_i .

Essa aproximação é dada pela Equação 51, em que a derivada de $f(x)$ em x_i é calculada pela razão entre a variação da função, $f(x_i) - f(x_{i-1})$, e a variação da variável x , $x_i - x_{i-1}$:

$$f'(x) = \left. \frac{df(x)}{dx} \right|_{x=x_i} \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}} \quad (51)$$

Este método é particularmente útil quando os dados à frente de x_i não estão disponíveis ou quando se deseja minimizar a influência de valores futuros na aproximação.

7.4 DERIVAÇÃO CENTRADA

A diferença central é uma técnica numérica utilizada para aproximar a derivada de uma função em um ponto, utilizando os valores da função em pontos ao redor. Ao contrário das diferenças progressivas ou regressivas, que consideram apenas pontos à frente ou atrás de x_i , a diferença central faz uso dos valores de $f(x)$ em ambos os lados de x_i , proporcionando uma aproximação mais precisa, especialmente para funções suaves.

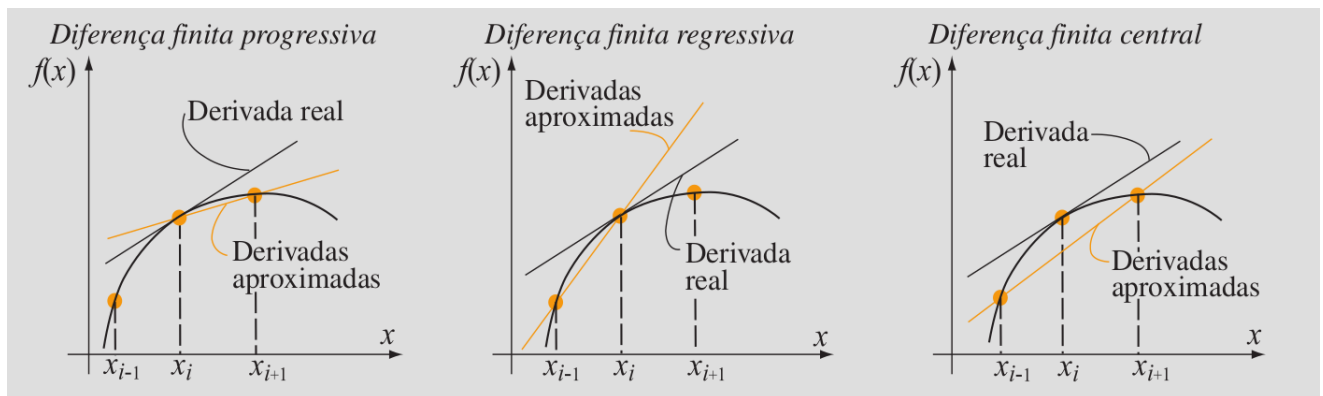
Essa abordagem é representada pela Equação 52, onde a derivada de $f(x)$ no ponto x_i é aproximada pela razão entre a diferença dos valores da função em x_{i+1} e x_{i-1} , e a distância entre esses pontos:

$$f'(x) = \left. \frac{df(x)}{dx} \right|_{x=x_i} \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{x_{i+1} - x_{i-1}} \quad (52)$$

Este método geralmente oferece uma melhor aproximação da derivada em comparação com as diferenças progressivas ou regressivas, principalmente quando a função é suficientemente suave no intervalo considerado.

Resumidamente, uma percepção gráfica da derivação numérica é apresentada na Figura 8. Note

Figura 8 – Equações de diferença para a derivada primeira



que, a depender do tipo de aproximação utilizada, os resultados apresentarão um erro de magnitude maior ou menor.

7.5 ERRO NA APROXIMAÇÃO DA DERIVAÇÃO

O erro em métodos de diferenças finitas está relacionado com o valor do incremento (h) entre pontos adjacentes. Enquanto um valor menor de h pode reduzir o erro de truncamento, valores muito pequenos podem amplificar erros de arredondamento, conforme demonstrado a seguir na Equação 53:

$$Erro_{relativo} = \left| \frac{f(x)_{aproximado} - f(x)_{real}}{f(x)_{real}} \cdot 100 \right| = f(x)\% \quad (53)$$

7.5.1 PERCEPÇÃO NUMÉRICA DO ERRO DE APROXIMAÇÃO

Considere a função $f(x) = x^3$. Calcule numericamente a derivada primeira no ponto $x = 3$, aplicando as fórmulas de diferenças finitas progressiva, regressiva e central, utilizando os pontos $x = 2$, $x = 3$ e $x = 4$. Então, compare os resultados com a derivada exata (analítica).

Solução

1. Aplicando-se a derivação analítica sobre $f(x)$ no ponto $x = 3$, temos:

$$\text{Solução analítica : } \begin{cases} f(x) = x^3 \\ f'(x) = 3 \cdot x^2 \\ f'(3) = 3 \cdot (3)^2 = 27 \\ f'(3) = 27 \end{cases}$$

2. Aplicando-se a derivação numérica no ponto $x = 3$, temos os resultados:

- a) Diferenciação finita progressiva, Equação 50: $\left. \frac{df(x)}{dx} \right|_{x=3} = \frac{f(4) - f(3)}{4 - 3} = \frac{4^3 - 3^3}{1} = 37$
- b) Diferenciação finita regressiva, Equação 51: $\left. \frac{df(x)}{dx} \right|_{x=3} = \frac{f(3) - f(2)}{3 - 2} = \frac{3^3 - 2^3}{1} = 19$
- c) Diferenciação finita central, Equação 52: $\left. \frac{df(x)}{dx} \right|_{x=3} = \frac{f(4) - f(2)}{4 - 2} = \frac{4^3 - 2^3}{2} = 28$

$$\text{Portanto, a derivação numérica : } \begin{cases} \text{progressiva : } f'(3) = 37 \\ \text{regressiva : } f'(3) = 19 \\ \text{central : } f'(3) = 28 \end{cases}$$

3. Analisando-se os erros na aproximação, conforme Equação 53, tem-se :

$$\text{Erros de aproximação: } \begin{cases} \text{progressiva : } Erro = \left| \frac{37-27}{27} \cdot 100 \right| = 37,04\% \\ \text{regressiva : } Erro = \left| \frac{19-27}{27} \cdot 100 \right| = 29,63\% \\ \text{central : } Erro = \left| \frac{28-27}{27} \cdot 100 \right| = 3,704\% \end{cases}$$

Neste caso, o erro aproximado é de 37,04% utilizando a derivação progressiva, 29,63% utilizando a derivação regressiva e de apenas 3,704% utilizando a derivação central.

7.6 SCRIPT BÁSICO PYTHON PARA DERIVADAS NUMÉRICAS

```

from autograd import grad
def der_num(x,y,p):
    if p<min(x) or p> max(x):
        print(f"\nPonto 'x={p}' de analise fora do dominio do intervalo!\n")
        return None,None,None

    elif p not in x:
        print(f"\nValor 'x={p}' esta fora da lista.\nUma interpolacao pode ser
            necessaria!\n")
        return None,None,None
    else:
        try:
            # Encontrar o indice de p na lista x
            i = x.index(p)
            dp,dr,dc = None,None,None
            if i==0:
                dp=( y[i+1] - y[i])/(x[i+1]-x[i])
            elif i==len(x)-1:
                dr=( y[i] - y[i-1])/(x[i]-x[i-1])
            else:
                dp=( y[i+1] - y[i])/(x[i+1]-x[i])
                dr=( y[i] - y[i-1])/(x[i]-x[i-1])
                dc=( y[i+1] - y[i-1])/(x[i+1]-x[i-1])
        finally:
            return dp,dr,dc

def Resultados(dp,dr,dc,p,f):
    df_dx = grad(f); der = df_dx(float(p))
    print('-'*50, f"\nRESULTADOS para o ponto x='{p}':\n", '-'*50, sep='')
    print("Analitica \tProgr\t \t\tCentral\t \tRegress")
    print(f" {der}\t \t {dp} \t {dc} \t {dr}")
    E=[]
    for derivada_num in [dp,dc,dr]:
        if derivada_num==None: E.append(None)
        else: E.append(round(abs(der-derivada_num)/der,2))
    print(f" Erro(%): \t {E[0]} \t\t\t {E[1]} \t {E[2]}")

if __name__=="__main__":
    f=lambda y:y**3
    x=[2,3,4,5,6];
    y=[f(i) for i in x]
    print(x);
    for p in [1,5, 2, 3, 6, 6.5]: # Pontos de analise
        dp,dr,dc= der_num(x,y,p)
        Resultados(dp,dr,dc,p,f)

```

7.7 DIFERENÇAS FINITAS POR SÉRIE DE TAYLOR

As fórmulas de diferenças finitas são amplamente utilizadas para aproximar derivadas de funções em pontos discretos. Utilizando expansões de série de Taylor, podemos deduzir fórmulas de diferenças finitas para derivadas de primeira e segunda ordem, aplicáveis em problemas de métodos numéricos.

CONCEITOS BASILARES DA SÉRIE DE TAYLOR

A forma genérica da série de Taylor de uma função $f(x)$ em torno de um ponto $x = a$ é dada por:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n, \quad (54)$$

em que $f^{(n)}(a)$ representa a n -ésima derivada de f avaliada em $x = a$.
ou, de maneira mais expandida,

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \cdots \quad (55)$$

7.7.1 APLICAÇÃO SÉRIE DE TAYLOR ÀS DIFERENÇAS FINITAS

Para uma função $f(x)$ suficientemente diferenciável em torno de um ponto x , a expansão em série de Taylor ao redor deste ponto pode ser expressa como:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \cdots \quad (56)$$

onde h representa um pequeno deslocamento a partir de x . Esta expansão fornece uma ferramenta poderosa para desenvolver aproximações de derivadas, ao manipularmos termos desta série para resolver para $f'(x)$ e $f''(x)$.

7.7.2 APROXIMAÇÃO PARA A DERIVADA PRIMEIRA

Consideremos a derivada primeira de uma função $f(x)$. Podemos obter aproximações para $f'(x)$ reescrevendo a expansão de Taylor para pontos adjacentes e então subtraindo ou somando essas expressões.

Fórmula de Diferença Progressiva

Expandindo $f(x+h)$ em série de Taylor, temos:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + O(h^3) \quad (57)$$

Ao resolvermos para $f'(x)$, obtemos a seguinte aproximação:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (58)$$

Esta é uma fórmula de diferença progressiva de primeira ordem para a derivada primeira de $f(x)$.

Fórmula de Diferença Central

Para melhorar a precisão, utilizamos a fórmula de diferença central, que envolve pontos de ambos os lados de x . Expandindo $f(x+h)$ e $f(x-h)$, temos:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + O(h^3) \quad (59)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2!}f''(x) - O(h^3) \quad (60)$$

Subtraindo estas equações, obtemos:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (61)$$

Esta é uma fórmula de diferença central de segunda ordem para a derivada primeira, com erro da ordem de $O(h^2)$.

7.7.3 APROXIMAÇÃO PARA A DERIVADA SEGUNDA

Para a derivada segunda de $f(x)$, somamos as expansões de Taylor para $f(x+h)$ e $f(x-h)$.

Fórmula de Diferença Central para a Derivada Segunda

Somando as expansões, obtemos:

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + O(h^4) \quad (62)$$

Resolvendo para $f''(x)$, temos a seguinte fórmula:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \quad (63)$$

Esta é uma fórmula de diferença central de segunda ordem para a derivada segunda de $f(x)$.

7.7.4 EXEMPLOS DE CÁLCULO NUMÉRICO

Exemplo 1: Aproximação da Derivada Primeira

Suponha que queremos aproximar a derivada da função $f(x) = \sin(x)$ em $x = 0.5$, com $h = 0.1$. Utilizando a fórmula de diferença progressiva, temos:

$$\begin{aligned} f'(0.5) &\approx \frac{f(0.5+0.1) - f(0.5)}{0.1} \\ &= \frac{\sin(0.6) - \sin(0.5)}{0.1} \\ &\approx \frac{0.5646 - 0.4794}{0.1} = 0.852 \end{aligned}$$

Exemplo 2: Aproximação da Derivada Segunda

Consideremos a mesma função $f(x) = \sin(x)$ e queremos calcular uma aproximação para $f''(0.5)$, com $h = 0.1$. Utilizando a fórmula de diferença central para a derivada segunda, obtemos:

$$\begin{aligned} f''(0.5) &\approx \frac{f(0.5 + 0.1) - 2f(0.5) + f(0.5 - 0.1)}{(0.1)^2} \\ &= \frac{\sin(0.6) - 2\sin(0.5) + \sin(0.4)}{0.01} \\ &= \frac{0.5646 - 2 \cdot 0.4794 + 0.3894}{0.01} \\ &\approx \frac{0.5646 - 0.9588 + 0.3894}{0.01} = -0.048 \end{aligned}$$

7.8 CONSIDERAÇÕES FINAIS

A derivação numérica possui técnicas poderosas para a resolução de problemas onde as abordagens analíticas não são viáveis. No entanto, o sucesso dessas técnicas depende de uma boa escolha do método e de parâmetros, como o passo h , como também, de uma compreensão dos erros associados a cada método para a obtenção de resultados confiáveis.

8 Integração Numérica

A integração numérica, assim como a derivação numérica, é ferramentas fundamentais no campo dos métodos numéricos. Elas permitem a aproximação de integrais de funções que não podem ser resolvidas analiticamente, ou cuja a solução analítica é demasiadamente trabalhosa. Neste capítulo, abordaremos os principais métodos de integração numérica, com ênfase em suas aplicações e limitações.

8.1 CONCEITUAÇÃO INTEGRAÇÃO NUMÉRICA

A integração numérica é uma ferramenta fundamental para calcular integrais analíticas, conforme revisado na Secção 4.4, definidas de funções, especialmente quando a solução analítica não está disponível ou é muito complexa.

- **Métodos de integração numérica:** São técnicas que fornecem aproximações para os valores de integrais definidas. Esses métodos são amplamente utilizados em diversas áreas da engenharia e ciências aplicadas, principalmente quando a solução exata da integral não pode ser obtida de maneira analítica.
- **Quando a integração numérica é útil:**
 - Quando a função $f(x)$ não é conhecida de forma explícita, mas são fornecidos apenas valores discretos de $f(x)$, como em uma tabela de dados experimentais.
 - Quando $f(x)$ é conhecida, mas sua forma é muito complicada, tornando impraticável a obtenção de uma primitiva ou solução analítica.
- **Ideia básica dos métodos de integração:** O conceito central da integração numérica é aproximar a função $f(x)$ por um polinômio, que se ajusta de forma satisfatória dentro de um intervalo $[a, b]$. A partir dessa aproximação, o cálculo da integral é reduzido à integração de um polinômio, tarefa que pode ser resolvida de forma relativamente simples e eficiente.

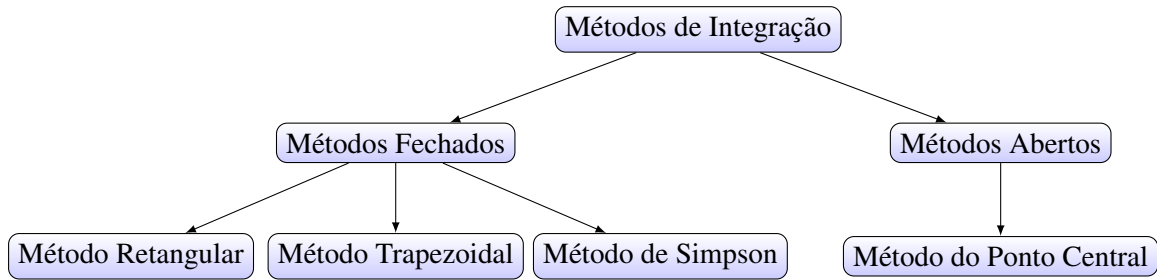
Existem diversos métodos para o cálculo numérico de integrais. Esses métodos deduzem fórmulas para obter o valor aproximado da integral utilizando pontos discretos do integrando. Em geral, eles são classificados em duas categorias principais:

- **Métodos fechados:** Utilizam valores da função em pontos que incluem as extremidades do intervalo de integração.
- **Métodos abertos:** Consideram pontos que não incluem as extremidades, sendo úteis quando os valores da função não estão disponíveis nas bordas do intervalo.

Esses métodos são amplamente empregados para resolver problemas práticos em que as funções envolvidas não podem ser integradas de forma exata, sendo a integração numérica uma solução eficiente para obter resultados aproximados com boa precisão. Na Figura 9 é apresentada uma percepção geral do métodos de integração.

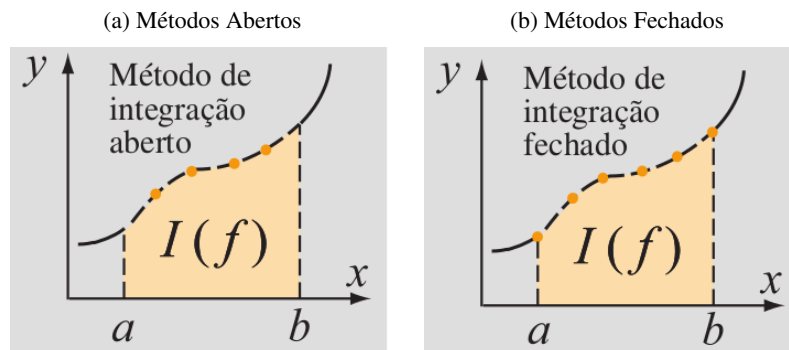
Graficamente, essa percepção é exibida nas figuras 10a e 10b.

Figura 9 – Métodos de interação



Fonte: Autor,(2024)

Figura 10 – Comparação dos métodos de integração



Fonte: GILAT, (2008)

8.2 MÉTODO DO RETÂNGULO

O Método do Retângulo aproxima a integral pela soma de áreas de retângulos sob a curva.

8.2.1 MÉTODO DO RETÂNGULO SIMPLES

A equação básica para a aproximação da integral de uma função $f(x)$ no intervalo $[a, b]$, pelo **Método do Retângulo Simples (MRS)**, é dada pela Equação 64:

$$I(f) = \int_a^b f(x)dx = f(a)(b-a) \approx f(b)(b-a) \quad (64)$$

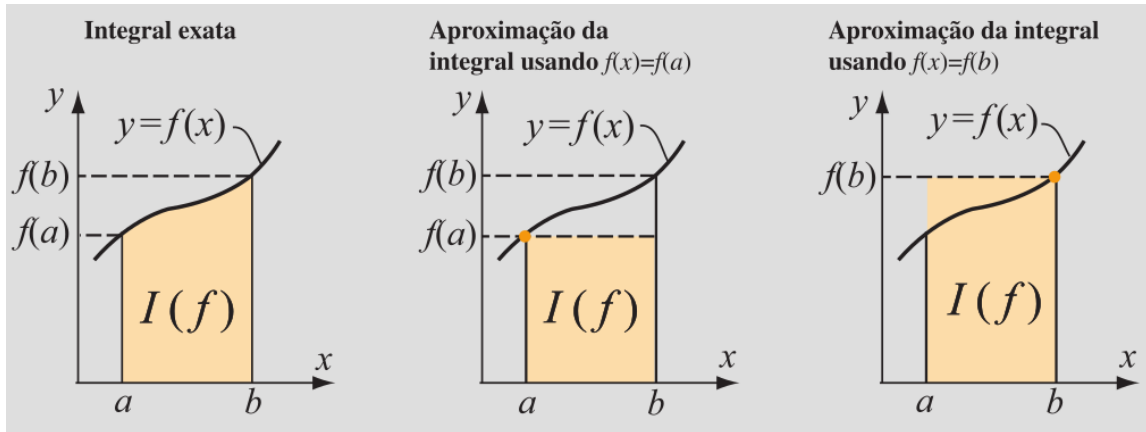
Essa aproximação do **MRS** é, contudo, suscetível de erros mais grosseiros, conforme por ser percebido na Figura 11.

Isso implica que para :
$$\begin{cases} f(x) = f(a), I_{aprox}(f) < I_{exata}(f), \text{ tendendo a } I_{aprox}(f) \ll I_{exata}(f) \\ f(x) = f(b), I_{aprox}(f) > I_{exata}(f), \text{ tendendo a } I_{aprox}(f) \gg I_{exata}(f) \end{cases}$$

8.2.2 MÉTODO DO RETÂNGULO COMPOSTO

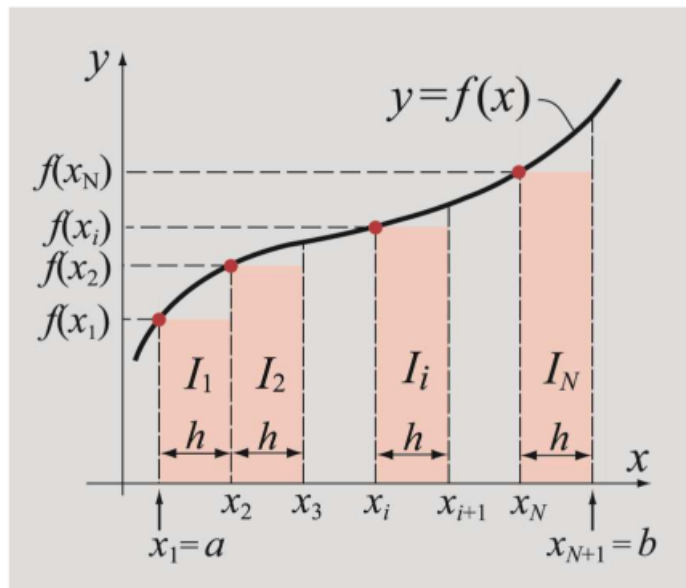
Visando reduzir o erro apresentado no **MRS**, no **Método do Retângulo Composto (MRC)** criam-se N subintervalos sobre os quais aplica-se o mesmo método, seguindo-se a soma do resultado do integral de cada intervalo, conforme apresentado na Figura 12 e definido na Equação 65.

Figura 11 – Método do Retângulo Simples



Fonte: GILAT,(2008)

Figura 12 – Método do Retângulo Composto



Fonte: GILAT,(2008)

$$I(f) = \int_a^b f(x)dx \approx h \sum_{i=1}^N f(x_i) \quad (65)$$

Note que a Equação 65 é aplicável quando os subintervalos têm a mesma largura h . Doutro modo, os vários intervalos permanecem desagrupados e o somatório é realizado individualmente intervalo-a-intervalo.

8.3 MÉTODO DO PONTO CENTRAL

O Método do Ponto Central aproxima a integral pela soma de áreas de retângulos sob a curva, utilizando o ponto médio do intervalo da área sob análise, em lugar dos pontos de extremos.

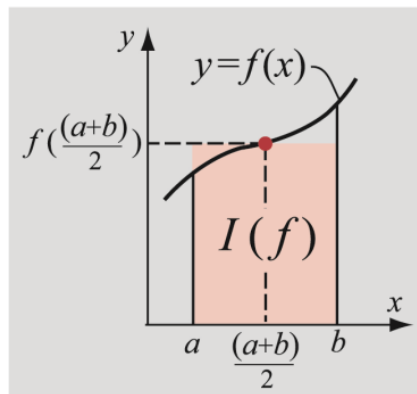
8.3.1 MÉTODO DO PONTO CENTRAL SIMPLES

A equação básica para a aproximação da integral pelo **Método do Ponto Central Simples (MPCS)** de uma função $f(x)$ no intervalo $[a, b]$ é dada pela Equação 66:

$$I(f) = \int_a^b f(x)dx = f\left(\frac{a+b}{2}\right)(b-a) \quad (66)$$

Essa aproximação é, contudo, suscetível de erros mais grosseiros, conforme por ser percebido na Figura 13.

Figura 13 – Método do Ponto Central Simples



Fonte: GILAT,(2008)

8.3.2 MÉTODO DO PONTO CENTRAL COMPOSTO

Visando reduzir o erro apresentado no **MPCS**, no **Método do Ponto Central Composto (MPCC)** criam-se N subintervalos sobre os quais aplica-se o mesmo método, seguindo-se a soma do resultado do integral de cada intervalo, conforme apresentado na Figura 14 e definido na Equação 67.

$$I(f) = \int_a^b f(x)dx \approx h \sum_{i=1}^N f(x_i) \quad (67)$$

Note que a Equação 67 é aplicável quando os subintervalos têm a mesma largura h . Doutro modo, os vários intervalos permanecem desagrupados e o somatório é realizado individualmente intervalo-a-intervalo.

8.4 MÉTODO DO TRAPÉZIO

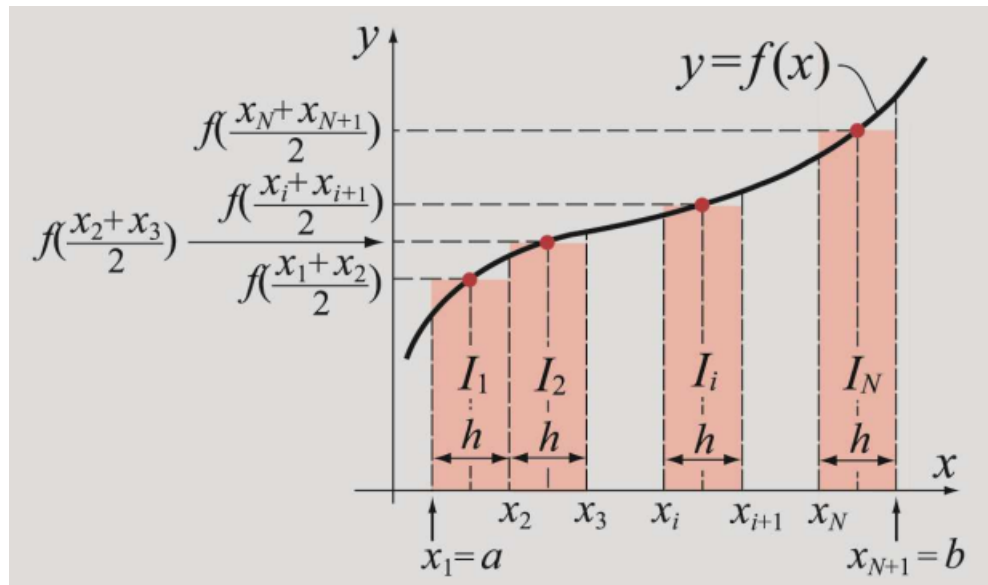
O Método do Trapézio aproxima a integral pela soma de áreas de trapézios sob a curva.

8.4.1 MÉTODO DO TRAPÉZIO SIMPLES

A equação básica para a aproximação da integral de uma função $f(x)$ no intervalo $[a, b]$, pelo **Método do Trapézio Simples (MTS)** Simples, é dada pela Equação 68:

$$\int_a^b f(x)dx \approx \frac{b-a}{2} [f(a) + f(b)] \quad (68)$$

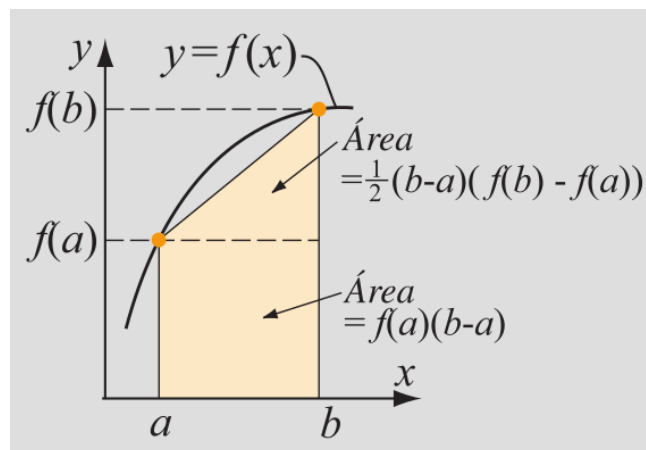
Figura 14 – Método do Ponto Central Composto



Fonte: GILAT,(2008)

Essa aproximação é, contudo, suscetível de erros mais grosseiros, conforme por ser percebido na Figura 15.

Figura 15 – Método do Trapézio Simples

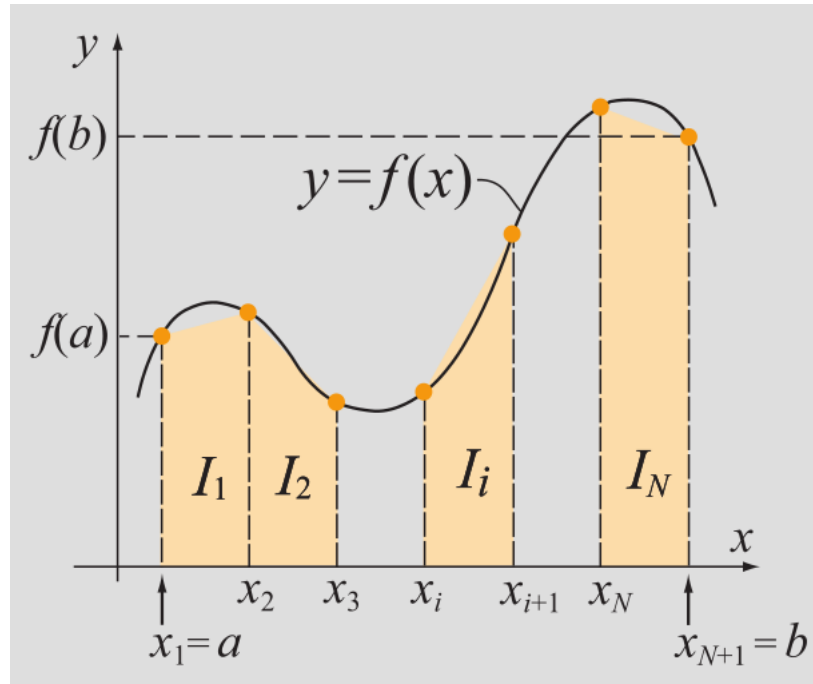


Fonte: GILAT,(2008)

8.4.2 MÉTODO DO TRAPÉZIO COMPOSTO

Visando reduzir o erro apresentado no MTS, no [Método do Trapézio Simples \(MTC\)](#) criam-se N subintervalos sobre os quais aplica-se o mesmo método, seguindo-se a soma do resultado do integral de cada intervalo, conforme apresentado na Figura 16 e definido na Equação 69.

Figura 16 – Método do Trapézio Composto



Fonte: GILAT,(2008)

Note que a aplicação do método trapezoidal em cada subintervalo $[x_i, x_{i+1}]$ resulta em:

$$I_i(f) = \int_{x_i}^{x_{i+1}} f(x)dx \approx \frac{[f(x_i) + f(x_{i+1})]}{2} (x_{i+1} - x_i)$$

Substituindo-se a aproximação trapezoidal, tem-se:

$$I(f) = \int_a^b f(x)dx \approx \frac{1}{2} \sum_{i=1}^N [f(x_i) + f(x_{i+1})] (x_{i+1} - x_i) \quad (69)$$

A Equação 69 é a equação geral do MTC. Neste caso, os subintervalos $[x_i, x_{i+1}]$ não precisam ser igualmente espaçados, podendo, portanto, cada um dos subintervalos pode ter uma largura diferente.

Contudo, se todos os subintervalos tiverem o mesmo tamanho, a referida expressão pode ser reduzida a uma fórmula útil para a programação, na qual a soma é expandida:

Para efeitos de linguagem de programação essa Equação pode ser representada como:

$$I(f) \approx \frac{h}{2} [f(a) + f(b)] + h \sum_{i=1}^N f(x_i),$$

caso h tenha larguras idênticas.

Note que a Equação 69 é aplicável quando os subintervalos têm a mesma largura h . Doutro modo, os vários intervalos permanecem desagrupados e o somatório é realizado individualmente intervalo-a-intervalo.

8.5 REGRA DE SIMPSON

A Regra de Simpson é uma técnica de aproximação numérica usada para calcular integrais definidas. Ela é especialmente útil quando a função $f(x)$ não pode ser integrada de forma analítica.

Existem duas variações principais da Regra de Simpson: a **Regra de Simpson 1/3** e a **Regra de Simpson 3/8**. Ambas se baseiam na aproximação da integral através de polinômios interpoladores de grau 2 ou 3.

8.5.1 REGRA DE SIMPSON 1/3

A Regra de Simpson 1/3 se baseia na aproximação da função $f(x)$ por um polinômio de segundo grau (parábola) dentro de um intervalo $[a, b]$. Esse método divide o intervalo de integração em dois subintervalos de igual comprimento e faz uso de três pontos igualmente espaçados, a , b , e o ponto médio $(a+b)/2$.

A Equação 70 define a Regra de Simpson 1/3::

$$I(f) = \int_a^b f(x)dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \quad (70)$$

Em que:

- a e b são os limites de integração;
- $f(a)$, $f(b)$ e $f\left(\frac{a+b}{2}\right)$ são os valores da função nos pontos a , b e no ponto médio.

Essa equação utiliza pesos 1, 4 e 1, sendo que a função avaliada no ponto médio recebe um peso maior, o que melhora a aproximação. A Regra de Simpson 1/3 é exata para polinômios de até segundo grau.

8.5.2 REGRA DE SIMPSON 3/8

A Regra de Simpson 3/8 se baseia na aproximação da função $f(x)$ por um polinômio cúbico (terceiro grau) dentro de um intervalo $[a, b]$. Diferente da Regra de Simpson 1/3, aqui o intervalo de integração é dividido em três subintervalos iguais e utiliza-se quatro pontos igualmente espaçados: a , $a+h$, $a+2h$ e b , onde $h = \frac{b-a}{3}$.

A Equação 71 define a Regra de Simpson 3/8 :

$$I(f) = \int_a^b f(x)dx \approx \frac{3(b-a)}{8} [f(a) + 3f(a+h) + 3f(a+2h) + f(b)] \quad (71)$$

Em que:

- a e b são os limites de integração;
- $h = \frac{b-a}{3}$ é a distância entre os pontos;
- $f(a)$, $f(a+h)$, $f(a+2h)$ e $f(b)$ são os valores da função nesses quatro pontos.

Diferente da Regra de Simpson 1/3, essa equação utiliza pesos 1, 3, 3 e 1. Assim como a Regra de Simpson 1/3, a Regra de Simpson 3/8 é exata para polinômios de grau até **três**, mas pode ser mais precisa em alguns casos, especialmente para funções que exibem maior curvatura.

8.6 SCRIPTS PYTHON PARA MÉTODOS INTEGRAÇÃO

8.6.1 INTEGRAÇÃO NUMÉRICA SIMPLES

O *script* em Python a seguir mostre como resultado um comparativo entre os métodos simples de integração numérica.

```
"""
Integracao via Metodos de retangulo, trapezio e ponto central simples
"""
f=lambda x: 97000*x/(5*x**2 + 570000)
a,b=40,93

I=[]
s=f(a)*(b-a) # Retangulo simples extremo a
I.append(s); s=0
s=f(b)*(b-a) # Retangulo simples extremo b
I.append(s); s=0
s=f((a+b)/2)*(b-a) # Ponto central
I.append(s); s=0
s=((f(a)+f(b))/2)*(b-a) # Trapezio Simples
I.append(s); s=0

M=["Retangulo simples altura 'a'", "Retangulo simples altura 'b'",
  'Ponto central', 'Trapezio Simples']
print()
for i in range(len(I)):
    print(round(I[i],4), ' >> Metodo:', M[i])
# Integral Analitica via biblioteca scipy
import scipy.integrate as integrate
g,e=integrate.quad(f,a,b)
print(f'\nSolucao analitica {round(g,4)}, via comando quad - scipy')
```

8.6.2 INTEGRAÇÃO NUMÉRICA COMPOSTA

O *script* em Python a seguir mostre como resultado um comparativo entre os métodos compostos de integração numérica.

```
import numpy as np
import scipy.integrate as integrate

def metod_retan(a,b,f,Ns:list=[10,100,1000]):
    ''' Metodos de trapezio composto com variacao da largura dos intervalos '''
    print("\nMetodo de retangulo composto:")
    for j in Ns:
        N=j #Total de intervalos
        h=(b-a)/N # Largura de cada intervalo
        x=np.arange(a, (b+h),h) # Intervalo particionado a:h:b
        s=0
        for i in range(N): # Retangulo composto
            s+=f(x[i])*h
```

```

    print(f'Solucao para {N} partes eh {round(s,4)}')

def metod_p_central(a,b,f,Ns:list=[10,100,1000]):
    ''' Metodos do ponto central composto com variacao da largura dos intervalos'''

    print("\nMetodo do Ponto Central Composto")
    for j in Ns:
        N=j #Total de intervalos
        h=(b-a)/N # Largura de cada intervalo
        x=np.arange(a, (b+h),h) # Intervalo particionado a:h:b
        s=0
        for i in range(N): # Ponto central composto
            s+=f((x[i+1] + x[i])/2)*h
        print(f'Solucao para {N} partes eh {round(s,4)}')

def metod_trapz(a,b,f,Ns:list=[10,100,1000]):
    ''' Metodos de trapezio composto com variacao da largura dos intervalos '''
    print("\nMetodo de trapezio composto:")
    for j in Ns:
        N=j #Total de intervalos
        h=(b-a)/N # Largura de cada intervalo
        x=np.arange(a, (b+h),h) # Intervalo particionado a:h:b
        s=0
        for i in range(N): # Trapezio composto
            s+=0.5*( f(x[i]) + f(x[i+1]))*h
        print(f'Solucao para {N} partes eh {round(s,4)}')

if __name__ == "__main__":

    f=lambda x: 97000*x/(5*x**2 + 570000)
    a,b = 40,93
    Ns=[10,100,1000,10000,100000]

    metod_retan(a,b,f,Ns)
    metod_p_central(a,b,f,Ns)
    metod_trapz(a,b,f,Ns)

    g,e=integrate.quad(f,a,b)
    print(f'\nSolucao analitica {round(g,4)}, via comando quad - scipy')

```

8.7 CONSIDERAÇÕES FINAIS

A integração numérica possui técnicas poderosas para a resolução de problemas onde as abordagens analíticas não são viáveis. No entanto, o sucesso dessas técnicas depende de uma boa escolha do método e de parâmetros, como o número de subintervalos para a integração, como também, de uma compreensão dos erros associados a cada método para a obtenção de resultados confiáveis.

9 Métodos de Interpolação

Os métodos de interpolação são fundamentais no campo dos métodos numéricos, permitindo a construção de funções aproximadas a partir de um conjunto discreto de pontos. Esses métodos são amplamente utilizados em engenharia para prever dados ou relacionar grandezas medidas experimentalmente.

A interpolação consiste em encontrar uma função que passe exatamente por um conjunto de pontos. Dado um conjunto de pontos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, podemos utilizar diferentes métodos para encontrar uma função $f(x)$ tal que $f(x_i) = y_i$ para todo i .

9.1 INTERPOLAÇÃO POLINOMIAL DE LAGRANGE

O método de interpolação polinomial busca encontrar um polinômio $P(x)$ de grau n que passe por todos os pontos dados. A forma mais comum é a interpolação de Lagrange, que define o polinômio interpolador é apresentada na Equação 72:

$$P(x) = \sum_{i=0}^n y_i L_i(x) \quad (72)$$

em que $L_i(x)$ são os polinômios de Lagrange definidos por:

$$L_i(x) = \prod_{\substack{0 \leq j \leq n \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

Exemplo

Dado o conjunto de pontos $(0, 1)$, $(1, 2)$ e $(2, 0)$, encontre o valor de y para $x = 1.5$

1) Calculando o polinômio de Lagrange, conforme Equação 72.

$$P(x) = 1 \cdot \frac{(x-1)(x-2)}{(0-1)(0-2)} + 2 \cdot \frac{(x-0)(x-2)}{(1-0)(1-2)} + 0 \cdot \frac{(x-0)(x-1)}{(2-0)(2-1)}$$

Simplificando:

$$P(x) = \frac{(x-1)(x-2)}{2} - 2 \cdot x(x-2)$$

2) Substituindo $x = 1.5$ no polinômio encontrado $P(x)$:

$$P(1.5) = \frac{(1.5-1)(1.5-2)}{2} - 2 \cdot 1.5(1.5-2)$$

Calculando os termos:

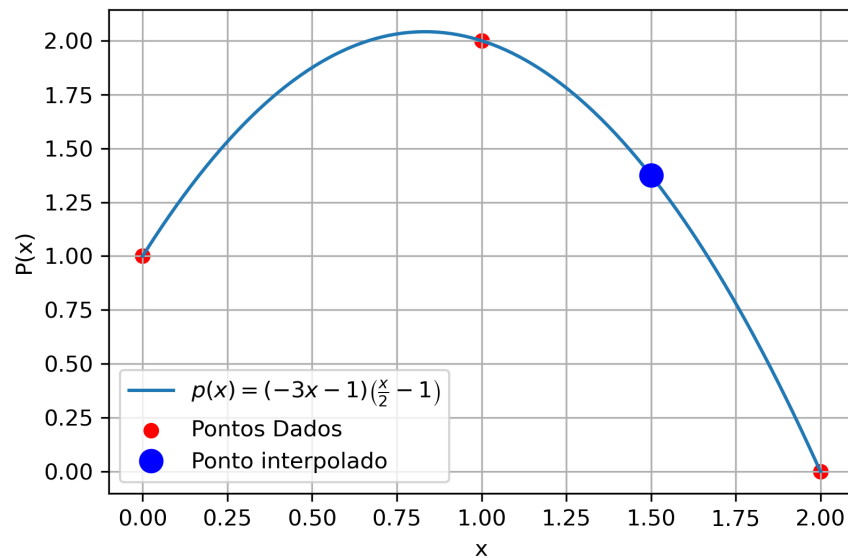
$$P(1.5) = \frac{(0.5)(-0.5)}{2} - 2 \cdot 1.5(-0.5)$$

$$P(1.5) = \frac{-0.25}{2} - 2 \cdot (-0.75)$$

$$P(1.5) = -0.125 + 1.5$$

$$P(1.5) = 1.375$$

Figura 17 – Interpolação de Lagrange



Fonte: Autor,(2024)

Assim, o valor interpolado para $x = 1.5$ usando o método de Lagrange é $P(1.5) = 1.375$.

Na Figura 17 é apresentada a percepção gráfica desta interpolação.

Observe na Figura 17 o polinômio $p(x)$. Note que o *script* computacional produziu, ocasionalmente, um $p(x)$ equivalente àquele imediatamente expresso a partir da Equação 72. Isso ocorre por efeito de simplificação, bem como das aproximações e ajustes da linguagem de programação utilizada, contudo, sem divergência nos resultados.

9.1.1 SCRIPT PYTHON PARA INTERPOLACAO DE LAGRANGE

```
import numpy as np
def PolIntLagr(x,y,p):
    ''' Exibe o resultado da interpolacao de lagrange um valor 'p' num pares de
        vetores X,Y.'''
    k=np.ones(len(x))
    for i in range(len(x)):
        for j in range(len(x)):
            if (i!=j):
                k[i]=k[i]*(p-x[j])/(x[i]-x[j])
    Yint=sum(y*k)
    return Yint

if __name__=="__main__":
    # Ponto de interpolacao e vetores de entrada x,y
    p=1.5;
    x=np.array([0,1,2])
    y=np.array([1,2,0])

    # Resultado da interpolacao do ponto 'p'
    Yint= PolIntLagr(x,y,p)
    print('\nA aproximacao encontrada para f(%.2f) = %.3f'%(p,Yint))
```

9.2 INTERPOLAÇÃO POR DIFERENÇAS DIVIDIDAS DE NEWTON

Outra abordagem é a interpolação por diferenças divididas de Newton, na qual o polinômio interpolador é descrito pela Equação 73:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots \quad (73)$$

em que $f[x_0, x_1, \dots, x_k]$ são as diferenças divididas calculadas a partir dos pontos.

Exemplo

Utilizando os mesmos pontos $(0, 1)$, $(1, 2)$ e $(2, 0)$ e o mesmo valor a ser interpolado $x = 1.5$, temos os seguintes valores de diferenças divididas:

1) Calculando o polinômio de Newton, conforme Equação 73.

$$f[x_0, x_1] = \frac{f(1) - f(0)}{1 - 0} = 1$$

$$f[x_0, x_1, x_2] = \frac{\frac{f(2) - f(1)}{2 - 1} - f[x_0, x_1]}{2 - 0} = \frac{\frac{0 - 2}{2 - 1} - 1}{2 - 0} = \frac{-2 - 1}{2} = -1.5$$

Portanto, o polinômio interpolador de Newton é:

$$P(x) = 1 + 1(x - 0) - 1.5(x - 0)(x - 1)$$

Simplificando:

$$P(x) = 1 + x - 1.5x(x - 1)$$

2) Substituindo $x = 1.5$ no polinômio encontrado $P(x)$:

$$P(1.5) = 1 + 1.5 - 1.5(1.5)(1.5 - 1)$$

$$P(1.5) = 1 + 1.5 - 1.5(0.75)$$

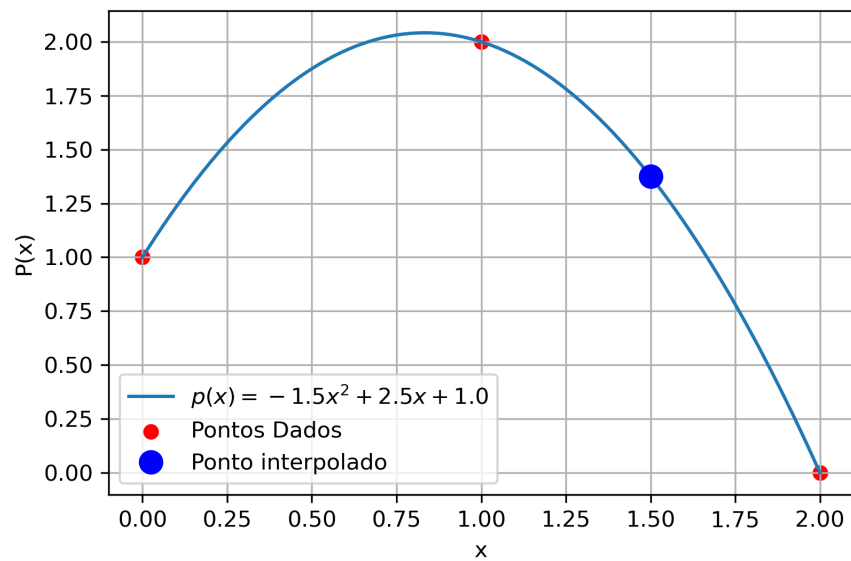
$$P(1.5) = 2.5 - 1.125$$

$$P(1.5) = 1.375$$

Na Figura 18 é apresentada a percepção gráfica desta interpolação.

Observe na Figura 18 o polinômio $p(x)$. Note que o *script* computacional produziu, ocasionalmente, um $p(x)$ equivalente àquele imediatamente expresso a partir da Equação 73. Isso ocorre por efeito de simplificação, bem como das aproximações e ajustes da linguagem de programação utilizada, contudo, sem divergência nos resultados.

Figura 18 – Interpolação de Newton



Fonte: Autor,(2024)

9.2.1 SCRIPT PYTHON PARA INTERPOLACAO DE NEWTON

```
import numpy as np
def PolInterNewton(x,y,p):
    ''' Exibe o resultado objetivo da interpolacao de Newton para um valor 'p' num
        pares de vetores X,Y.
    '''
    a,s= [y[0]],y # s eh vetor das divisoes
    for i in range(len(x)-1):
        y=s # Atualiza o vetor y
        s=(y[1:]-y[:-1])/(x[(1+i):]-x[:- (1+i)]) # Reduz o vetor x
        a.append(s[0])
        xn,Yint=1,a[0]
        for k in range(1,len(x)):
            xn=xn*(p - x[k-1])
            Yint=Yint+a[k]*xn

        return Yint

if __name__=="__main__":
    # Ponto de interpolacao e vetores de entrada x,y
    p=1.5;
    x=np.array([0,1,2])
    y=np.array([1,2,0])

    # Resultado da interpolacao do ponto 'p'
    Yint= PolInterNewton(x,y,p)
    print('\nA aproximacao encontrada para f(%.2f) = %.3f'%(p,Yint))
```

9.3 COMPARATIVO ENTRE OS MÉTODOS DE INTERPOLAÇÃO

No método de Lagrange, o polinômio é construído a partir de uma combinação de polinômios base, em que cada um deles é responsável por garantir que o polinômio completo passe pelos pontos conhecidos. Esse método tende a ser mais direto para uma quantidade fixa de pontos, porém pode ser mais suscetível a erros numéricos quando o número de pontos aumenta, especialmente devido ao comportamento das funções base.

Por outro lado, o método de Newton constrói o polinômio de forma incremental, utilizando diferenças divididas entre os pontos de dados. Esse processo permite adicionar novos pontos ao polinômio sem a necessidade de recalcular toda a expressão, o que pode ser vantajoso em certos casos, como em aplicações em que se deseja ajustar novos dados progressivamente. No entanto, o cálculo das diferenças divididas pode levar a resultados diferentes, especialmente quando há mais sensibilidade a arredondamentos numéricos.

Em resumo, ambos os métodos geram polinômios que interpolam os mesmos pontos, mas a escolha de qual utilizar pode depender da aplicação. O método de Lagrange é simples e eficiente para conjuntos de pontos pequenos e fixos, enquanto o método de Newton oferece flexibilidade e eficiência quando há necessidade de ajustes dinâmicos ou de manipulação incremental dos dados.

10 Ajuste de curvas

Nesta capítulo, abordamos com maior rigor matemático os principais métodos de ajuste de curvas utilizados em problemas de engenharia e ciências aplicadas. Tais métodos visam encontrar uma função $f(x)$ que represente, com erro mínimo, um conjunto de pontos experimentais (x_i, y_i) , $i = 1, 2, \dots, n$.

10.1 REGRESSÃO LINEAR

A regressão linear simples procura ajustar uma função da forma:

$$f(x) = a_0 + a_1x$$

O objetivo é minimizar a soma dos quadrados dos resíduos:

$$S(a_0, a_1) = \sum_{i=1}^n (y_i - a_0 - a_1x_i)^2$$

Para minimizar S , derivam-se as equações normais:

$$\begin{cases} \frac{\partial S}{\partial a_0} = -2 \sum_{i=1}^n (y_i - a_0 - a_1x_i) = 0 \\ \frac{\partial S}{\partial a_1} = -2 \sum_{i=1}^n x_i (y_i - a_0 - a_1x_i) = 0 \end{cases}$$

Resolvendo o sistema, obtemos:

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad a_0 = \frac{1}{n} (\sum y_i - a_1 \sum x_i)$$

10.2 REGRESSÃO POLINOMIAL

Para dados que não seguem uma tendência linear, aplica-se a regressão polinomial. O modelo é um polinômio de grau m :

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$$

O critério de mínimos quadrados minimiza:

$$S(a_0, \dots, a_m) = \sum_{i=1}^n \left[y_i - \sum_{j=0}^m a_j x_i^j \right]^2$$

Derivando em relação a cada a_k e igualando a zero, obtém-se um sistema linear com $m+1$ equações, conhecido como **equações normais**:

$$\sum_{i=1}^n x_i^k y_i = \sum_{j=0}^m a_j \sum_{i=1}^n x_i^{j+k}, \quad \text{para } k = 0, 1, \dots, m$$

Este sistema pode ser resolvido por métodos numéricos como eliminação de Gauss ou decomposição LU.

10.3 SPLINE LINEAR

O *spline linear* consiste em ajustar, entre cada par consecutivo de pontos (x_i, y_i) e (x_{i+1}, y_{i+1}) , uma função linear da forma:

$$S_i(x) = a_i x + b_i, \quad x \in [x_i, x_{i+1}]$$

Os coeficientes são obtidos impondo que:

$$\begin{cases} S_i(x_i) = y_i \\ S_i(x_{i+1}) = y_{i+1} \end{cases} \Rightarrow \begin{cases} a_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \\ b_i = y_i - a_i x_i \end{cases}$$

O spline linear é contínuo em $[x_1, x_n]$, mas suas derivadas não são contínuas nos pontos de junção.

10.4 SPLINE QUADRÁTICO

Um *spline quadrático* ajusta um polinômio de grau 2 em cada intervalo $[x_i, x_{i+1}]$:

$$S_i(x) = a_i x^2 + b_i x + c_i$$

São impostas as seguintes condições:

- Interpolação: $S_i(x_i) = y_i$ e $S_i(x_{i+1}) = y_{i+1}$; - Continuidade da primeira derivada: $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$; - Uma condição adicional para fechamento do sistema, como derivada nula no primeiro ponto: $S''_1(x_1) = 0$.

Essas condições geram um sistema linear com $3(n-1)$ equações para determinar os coeficientes a_i , b_i e c_i .

10.5 SPLINE CÚBICO

O *spline cúbico* é o mais utilizado, por garantir suavidade até a segunda derivada. Em cada subintervalo $[x_i, x_{i+1}]$ define-se:

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

As condições impostas são:

- **Interpolação:** $S_i(x_i) = y_i$ e $S_i(x_{i+1}) = y_{i+1}$;
- **Continuidade da primeira derivada:** $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$;
- **Continuidade da segunda derivada:** $S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$;
- **Condições de contorno:** pode-se usar derivadas nulas nas extremidades (spline natural), ou derivadas conhecidas (spline clamped).

O sistema resultante possui $4(n-1)$ equações e é resolvido eficientemente por métodos como o algoritmo da matriz tridiagonal.

Na Tabela 3 é apresentado um comparativo basilar entre os métodos.

Tabela 3 – Comparativa entre Métodos de Ajuste de Curvas

Método	Forma do Modelo	Vantagens	Desvantagens	Complexidade
Regressão Linear	$y = a_0 + a_1x$	Simples, eficiente para dados com tendência linear	Inadequado para relações não lineares	$\mathcal{O}(n)$
Regressão Polinomial	$y = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$	Ajusta curvas mais complexas com flexibilidade	Risco de sobreajuste e instabilidade para graus altos	$\mathcal{O}(n^2)$
Spline Linear	$S_i(x) = a_ix + b_i$ por intervalo	Simples, computacionalmente leve, contínuo	Descontinuidade na derivada	$\mathcal{O}(n)$
Spline Quadrática	$S_i(x) = a_ix^2 + b_ix + c_i$ por intervalo	Continuidade da primeira derivada	Menos comum, mais cálculo que o linear	$\mathcal{O}(n)$
Spline Cúbica	$S_i(x) = a_ix^3 + b_ix^2 + c_ix + d_i$ por intervalo	Alta suavidade (até segunda derivada contínua)	Mais complexa, requer resolução de sistema linear	$\mathcal{O}(n)$

Fonte: Autor,(2025)

Aplicações de Engenharia

1. Seja o conjunto de dados: $x = \{2, 5, 6, 8, 9, 13, 15\}$, $y = \{7, 8, 10, 11, 12, 14, 15\}$.

-

use a regressão linear por mínimos quadrados para determinar os coeficientes m e b da função

$$y = mx + b$$

que melhor se ajusta aos dados.

Determinar o erro global.

- (JRC) Adicionalmente:

(A) Encontre o valor para $x = 7.25$.

(B) Trace um gráfico com a função encontrada e determine o valor estimado correspondente.

REFERÊNCIAS BIBLIOGRÁFICAS

CHAPRA, S. C.; CANALE, R. P. **Métodos Numéricos para Engenharia-7ª Edição**. [S.l.]: McGraw Hill Brasil, 2016.

COSTA, R. F. **8 alternativas open sources para o MatLab**. [s.n.], 2017. [Online; accessed 20-Dezembro-2019]. Disponível em: <<https://www.linuxdescomplicado.com.br/2017/03/8-alternativas-open-sources-para-o-matlab.html>>.

GILAT, A.; SUBRAMANIAM, V. **Métodos numéricos para engenheiros e cientistas: uma introdução com aplicações usando o MATLAB**. Porto Alegre - RS: Bookman, 2008. ISBN 978-85-7780-297-5.

HUNT, J. **A beginners guide to Python 3 programming**. [S.l.]: Springer, 2019.

MATHEWS, J. H. et al. **Métodos numéricos con Matlab**. [S.l.]: Prentice Hall, 2000. v. 2.

PINTO, J. C. **Métodos numéricos em problemas de engenharia química**. [S.l.]: E-papers, 2001.

SILVA, W. M. da et al. **Uma Alternativa Robusta, Rápida e Gratuita para Pesquisadores que Utilizam MATLAB**. s.l: [s.n.], 2014. Disponível em: <https://www.researchgate.net/profile/Flavio-Rossini/publication/324089118_Uma_Alternativa_Robusta_Rapida_e_Gratisita_para_Pesquisadores_que_Utilizam_MatLabr/links/5abd06860f7e9bfc0457a66d/Uma-Alternativa-Robusta-Rapida-e-Gratisita-para-Pesquisadores-que-Utilizam-MatLabr.pdf>. Acesso em: Out. 2023.